

ClipMind AI

Turn any video into a transcript, a study-grade summary and searchable key moments

Software Project Documentation & Technical Reference

A full-stack, role-based AI SaaS platform
Django REST Framework · Celery · PostgreSQL · Redis · React 19 · Docker

Document type	Final project report & technical reference
Repositories	<code>clip_mind_AI</code> (frontend) · <code>clip_mind_AI_Backend</code> (backend)
Backend	Django 5.2.16 · DRF 3.17.1 · Celery 5.6.3 · PostgreSQL 16
Frontend	React 19.2 · Vite 8 · Tailwind CSS 4 · React Router 7
AI providers	Groq (Whisper large-v3 + GPT-OSS-120B) → Google Gemini fallback
API surface	40 documented REST endpoints under <code>/api/v1/</code>
Automated tests	142 tests — all passing
Deployment	5-service Docker Compose stack (verified running)

All content in this document was derived directly from the project source code, configuration files and a live, running instance of the application. Every screenshot is a real capture of the deployed UI.

Table of Contents

1. Project Overview

Problem statement · solution · objectives · scope · user roles

2. Technology Stack

Backend, frontend, infrastructure and AI dependencies with exact versions

3. System Architecture

Component diagram · request lifecycle · runtime topology

4. Backend Implementation

Project layout · Django apps · core layer · services · settings

5. Authentication & Role-Based Access Control

JWT lifecycle · permission classes · capability matrix

6. Database Design

Entity-relationship diagram · table reference · migrations

7. The AI Processing Pipeline

Celery task stages · provider abstraction · fallback · prompt design

8. REST API Reference

Response envelope · endpoint tables · worked request/response examples

9. Frontend Implementation

Routing · state contexts · API client · role-driven navigation

10. Application Walkthrough (UI Screenshots)

Every major screen captured from the running application

11. Security Implementation

Transport, authentication, authorisation, upload, media and abuse controls

12. Deployment & Operations

Docker images · Compose topology · entrypoint · nginx · health probes

13. Configuration Reference

Environment variables and their effect

14. Testing & Validation

Test suite structure · execution evidence · AI output quality checks

15. Limitations, Design Trade-offs & Future Work

16. Appendix — Repository Inventory

How this document was produced

Every technical statement here was verified against the repository itself: models, serialisers, views, URL configuration, Celery tasks, permission classes, Dockerfiles and the Compose file. Endpoint lists were cross-checked against the OpenAPI schema the running server generates at `/api/schema/`. Test counts and outcomes come from an actual execution of `manage.py test` inside the running backend container. The screenshots in Section 10 were captured from the live application at `http://localhost:3000` with a real PostgreSQL database containing processed videos, AI summaries and engagement data.

1. Project Overview

1.1 Problem statement

Video is now the dominant medium for lectures, tutorials and knowledge sharing, but it is the least navigable one. A learner who needs a single definition from a ninety-minute lecture has no option but to scrub through a timeline. Educators cannot tell which parts of a recording students actually watched. Content is effectively opaque: it cannot be searched, skimmed, quoted or summarised without a human watching it end to end.

1.2 Solution

ClipMind AI converts a video — either an uploaded file or a YouTube link — into four machine-generated artefacts, and then builds a role-aware learning platform on top of them:

- **A timestamped transcript** produced by Whisper large-v3 speech-to-text, with automatic language detection and an optional user-supplied language hint.
- **A structured, study-grade summary** — eight distinct sections including a short overview, a multi-paragraph detailed summary, bullet takeaways, chapters, topics, action items, a glossary and exam-style question/answer pairs.
- **Key moments** — AI-identified highlights, each with a title, an explanation of why it matters, a timestamp and a thumbnail rendered from that exact frame of the video.
- **A locally stored, compressed, seekable copy of the media**, so the transcript and the highlights can be played back inside the product rather than linking out.

Around those artefacts the platform layers cross-video spoken-word search, sharing (private, class-scoped or public-link), bookmarks, learning history, educator-generated learning materials, on-demand translation of all AI output into any language, engagement analytics and a full administrator console.

1.3 Objectives delivered

Objective	How it is met in the implementation
Dual ingestion	Direct file upload (<code>POST /videos/upload</code>) and YouTube URL submission (<code>POST /videos/youtube</code>) converge on one Celery pipeline after the source is localised.
Accurate multilingual transcription	Groq Whisper <code>large-v3</code> in <code>verbose_json</code> mode returns text plus per-segment timings; language is auto-detected or forced from a validated ISO-639-1 hint.
Study-grade summarisation	A single JSON-mode LLM call returns eight named sections; the response is normalised so every key is always present.
Navigable highlights	Key moments are persisted with clamped timestamps, and FFmpeg renders a JPEG thumbnail per moment.
Search inside speech	Postgres filters transcripts by content; matching segments are then extracted with their start times so a result click seeks the player.
Four distinct user roles	Content Creator, Learner, Educator and Administrator, enforced by dedicated DRF permission classes and mirrored in the React navigation.
Resilience	If any Groq call fails, the provider manager transparently retries the same operation on Google Gemini.
Reproducible deployment	A five-service Docker Compose stack with health checks, volumes, an idempotent endpoint and an automatically seeded administrator.

1.4 User roles and capabilities

The role model is the organising principle of the whole system. It is defined once in `apps/accounts/models.py` and enforced independently on the server (permission classes) and on the client (route guards and navigation filtering).

Role	DB value	Capabilities in this implementation
Content Creator	<code>content_creator</code>	Upload files / YouTube links, run the AI pipeline, edit transcripts, build learning materials, share content, view content insights, download transcripts and summaries.
Learner	<code>learner</code>	Browse the shared library, read transcripts and summaries, jump to key moments, search spoken words, bookmark highlights, keep a learning history. cannot upload
Educator	<code>educator</code>	Everything a Content Creator can do, plus classroom analytics: per-student engagement, completion tracking and a 14-day engagement series across their own lectures.
Administrator	<code>admin</code>	Operates the platform: user and role management, activity monitoring, content moderation, live AI job monitoring, storage utilisation, platform settings, contact-message triage and the audit trail. deliberately cannot author content

Design decision worth noting. The Administrator is modelled strictly as an *operator*, not a super-user. `UPLOADER_ROLES` and `AUTHOR_ROLES` both exclude `admin`, so an administrator can delete or moderate somebody else's video but cannot upload one, edit a transcript, create learning materials or publish a share on another user's behalf. Ten dedicated tests (`AdminIsOperatorNotAuthorTests`) exist purely to hold that boundary in place.

1.5 Scope boundaries

The following are explicitly *not* part of the implemented system, and are documented here so the reader is not left guessing: there is no WebSocket layer (progress is delivered by lightweight REST polling), no payment or subscription handling, no Google OAuth flow (the `login_provider` field exists but only the e-mail provider is wired), no e-mail verification workflow, and no CDN or object-storage backend (media is served from the local filesystem through a range-capable Django view).

2. Technology Stack

All versions below are the pinned versions in `requirements.txt` and `package.json`, not approximations.

2.1 Backend

Package	Version	Role in the system
Django	5.2.16	Web framework, ORM, migrations, admin site, signing utilities.
djangorestframework	3.17.1	API layer: <code>APIView</code> s, serialisers, permissions, pagination, throttling.
djangorestframework-simplejwt	5.5.1	JWT issuing, rotation and refresh-token blacklisting.
drf-spectacular	0.29.0	OpenAPI 3.0 schema generation plus Swagger UI and ReDoc.
django-cors-headers	4.9.0	Cross-origin access for the SPA, with credentials enabled.
python-decouple	3.8	Environment-variable configuration; no secret is hardcoded.
psycopg2-binary	2.9.12	PostgreSQL driver.
celery	5.6.3	Distributed task queue running the AI pipeline off the request thread.
redis	8.0.1	Celery broker/result backend (DB 0) and Django cache (DB 1).
groq	1.5.0	Primary AI provider SDK — Whisper STT and LLM chat completions.
google-generativeai	0.8.6	Fallback AI provider SDK (Gemini).
yt-dlp	2025.6.30	YouTube download, using the <code>android</code> player client to avoid app restrictions.
imageio-ffmpeg	0.5.1	Ships a portable FFmpeg binary for compression, audio extraction and thumbnails.
pillow	12.3.0	Image handling for <code>ImageField</code> (thumbnails, profile images).
gunicorn	23.0.0	Production WSGI server (3 workers, 120 s timeout).
whitenoise	6.7.0	Serves collected static files (admin + Swagger assets) straight from Gunicorn.

2.2 Frontend

Package	Version	Role in the system
react / react-dom	19.2.7	UI runtime; the app is function components plus hooks throughout.
react-router-dom	7.18.1	Client-side routing, nested layout routes and route guards.
axios	1.18.1	HTTP client with request/response interceptors for JWT attach and silent refresh.
tailwindcss + @tailwindcss/vite	4.3.2	Utility-first styling; the entire dark theme is Tailwind classes, no CSS framework overrides.
framer-motion	12.42.2	Page and card transitions, modal animation.
vite	8.1.1	Dev server and production bundler.
eslint (+ react-hooks, react-refresh)	10.6.0	Static analysis; <code>npm run lint</code> .

Production bundle actually built from this source: `index-CK6_01Q3.js` ≈ 563 KB and `index-DI3mUBDy.css` ≈ 50 KB, served by nginx with immutable caching on hashed assets.

2.3 Infrastructure

Component	Image / tool	Purpose
Relational database	postgres:16-alpine	All persistent state; 27 tables including Django internals.
Broker & cache	redis:7-alpine	Celery transport (DB 0) and Django cache for translations and reset tokens (DB 1).
API server	python:3.12-slim	Gunicorn + Django; FFmpeg, libpq5 and curl installed at image build.
Worker	same image	Runs <code>celery -A config worker --concurrency=2</code> ; shares the media volume.
Web server	nginx:1.27-alpine	Serves the built SPA with history fallback and gzip.
Local Redis alternative	Memurai	Documented Windows-native Redis-protocol server for non-Docker development.

3. System Architecture

3.1 Component overview

ClipMind AI is a decoupled two-repository system. The React SPA is a pure API client: it holds no server-rendered state and talks exclusively to versioned JSON endpoints. The Django service is split into two runtime roles that share one codebase and one database — a synchronous API process and an asynchronous worker process — because the AI pipeline routinely takes minutes and must never block an HTTP request.

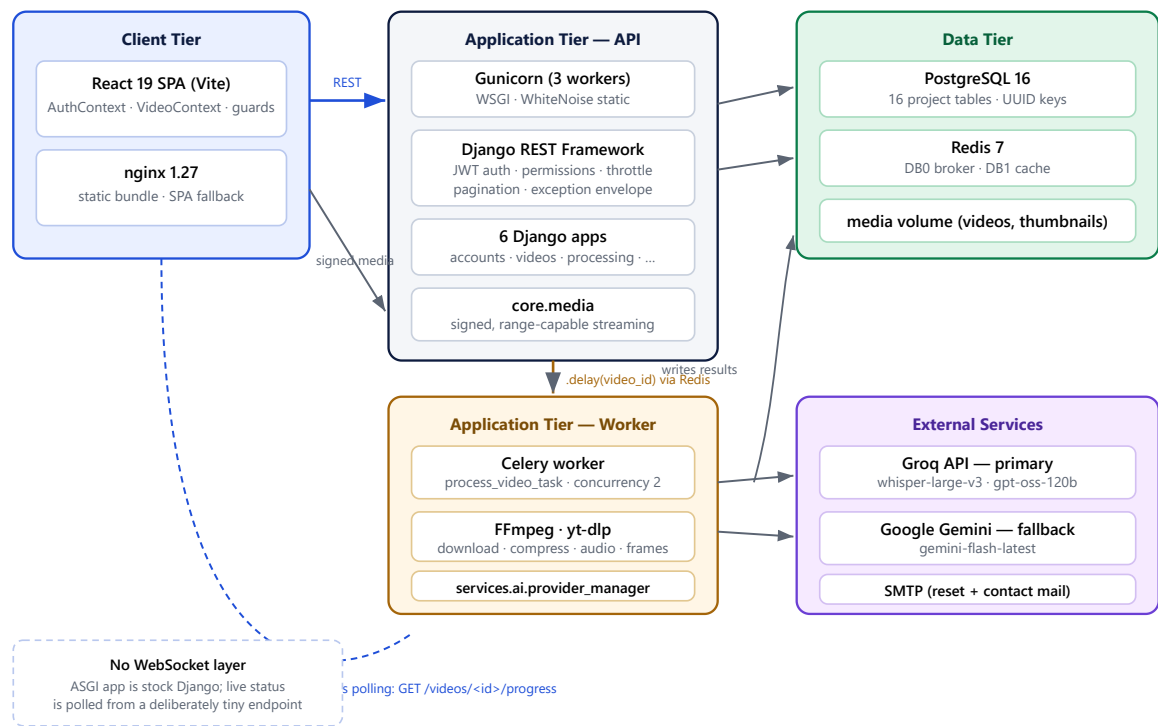


Figure 3.1 — Component architecture. Solid arrows are synchronous calls; the dashed path is the client's progress poll.

3.2 Request lifecycle

A representative authenticated request travels through the following chain, in order:

1. **nginx** (production only) serves `index.html`; all API traffic goes directly to the Django origin, so nginx is not a reverse proxy in this deployment.
2. **SecurityMiddleware** applies HSTS, content-type-nosniff and the HTTPS redirect when `DEBUG=False`.
3. **WhiteNoise** intercepts `/static/` requests and short-circuits them.
4. **CorsMiddleware** validates the `Origin` against `CORS_ALLOWED_ORIGINS`.
5. **DRF** resolves the view, runs `JWTAuthentication`, then the view's `permission_classes`, then throttling.
6. The view delegates to a **service module** (`AuthService`, `apps.videos.services`) rather than embedding logic, and returns through the shared `success_response` / `error_response` helpers.
7. Any exception is caught by `core.exception_handler.custom_exception_handler` and re-shaped into the same envelope, so the client never has to parse two error formats.

3.3 Why polling instead of WebSockets

`config/asgi.py` exposes the stock Django ASGI application with an explicit comment stating that no Channels layer is required. Instead, `VideoContext` polls `GET /api/v1/videos/<id>/progress` every three seconds, and only while the active video is in `pending` or `processing`. That endpoint returns four scalar fields (`status`, `progress`, `current_step`, `error_message`) and deliberately does not re-serialise the transcript, summary or key moments. The full record is fetched exactly once, when the status reaches a terminal state. This keeps the deployment to a single WSGI process type while still giving the user a live progress bar.

4. Backend Implementation

4.1 Project layout

```
clip_mind_AI_Backend/
├── config/                # Django project package
│   ├── settings.py       # single settings module, env-driven via python-decouple
│   ├── urls.py           # root URLconf — everything under /api/v1/
│   ├── celery.py         # Celery app, CELERY_ namespace, task autodiscovery
│   └── asgi.py wsgi.py
├── apps/
│   ├── accounts/         # User, ActivityLog, AuditLog, ContactMessage, PlatformSetting
│   │   ├── models.py     # serializers.py services.py views.py urls.py
│   │   ├── admin_views.py admin_urls.py # the whole Administrator module
│   │   ├── contact_views.py # public, throttled contact endpoint
│   │   └── migrations/   # 5 migrations incl. a legacy-role data migration
│   ├── videos/           # Video, VideoTranslation, VideoShare, VideoView,
│   │   ├──               # Bookmark, LearningHistory + all video endpoints
│   │   ├── services.py   # translation cache, visibility querysets, share creation
│   │   └── management/commands/cleanup_orphan_media.py
│   │       └── migrations/ # 6 migrations
│   ├── processing/tasks.py # the Celery AI pipeline (no models of its own)
│   ├── transcripts/      # Transcript model (2 migrations)
│   ├── summaries/       # Summary, KeyMoment, LearningMaterial (3 migrations)
│   └── analytics/views.py # 3 read-only aggregate endpoints (no models)
├── core/                # cross-cutting infrastructure, imported by every app
│   ├── responses.py      # success/error/created envelope helpers
│   ├── permissions.py    # 7 composable DRF permission classes
│   ├── exception_handler.py # global DRF exception → envelope
│   ├── exceptions.py     # 7 typed APIException subclasses
│   ├── media.py          # signed + range-capable media serving
│   ├── pagination.py     # health.py logging_service.py
│   ├── services/ai/      # provider abstraction
│   │   ├── base.py       # BaseAIProvider ABC — 4 abstract methods
│   │   ├── groq_provider.py gemini_provider.py provider_manager.py
│   └── utils/validators.py # upload, URL, language and password validation
├── Dockerfile docker-compose.yml entrypoint.sh requirements.txt
└── DEPLOYMENT.md RUNNING.md TESTING.md
```

4.2 The `core` layer

Rather than repeating boilerplate in every view, six shared modules define behaviour once. This is the main reason the view code reads cleanly.

Module	Responsibility and notable detail
core/responses.py	Three helpers produce the only three response shapes in the system. Every success carries <code>{success, message, data}</code> ; every failure carries <code>{success, message, errors}</code> .
core/exception_handler.py	Wraps DRF's default handler, flattens validation errors, logs status/path/message, and converts anything unhandled into a 500 in the same envelope so the SPA never sees a raw traceback shape.
core/permissions.py	Seven classes: <code>IsAdmin</code> , <code>IsEducator</code> , <code>IsContentCreator</code> , <code>CanUpload</code> , <code>CanAuthorContent</code> , <code>IsOwnerOrAdmin</code> , <code>IsOwnerOrReadOnlyShared</code> . Access rules are declarative on the view, so they are auditable in one file.
core/pagination.py	<code>StandardPagination</code> : 20 per page, <code>?page_size=</code> up to 100.
core/media.py	Signed-token minting (<code>TimestampSigner</code> , 12-hour TTL, bound to the exact file path), path-traversal guard, three-way authorisation, and HTTP <code>Range</code> support returning <code>206 Partial Content</code> so <code><video></code> seeking works.
core/logging_service.py	<code>record_activity()</code> and <code>record_audit()</code> . Both are deliberately failure-tolerant — a logging failure must never break the request that triggered it — and <code>client_ip()</code> honours <code>X-Forwarded-For</code> .
core/health.py	<code>/health/</code> executes <code>SELECT 1</code> and a cache write/read-back, returning <code>200 healthy</code> or <code>503 degraded</code> . Used by the container health check.

4.3 Logging configuration

Four rotating file handlers (10 MB × 5 backups each) split output by concern: `logs/api.log`, `logs/ai.log`, `logs/celery.log` and `logs/errors.log`, each also mirrored to the console so `docker compose logs` stays useful. Loggers are named `api`, `ai`, `celery`, `django` and `utils`, and every one has `propagate=False` to prevent duplicate records.

4.4 Service layer

Service	Contents
apps.accounts.services.AuthService	Login (with active-account check), logout via refresh-token blacklisting, rotation-aware refresh, password reset token issue/consume, password change. All views for authentication are thin wrappers over this class.
apps.videos.services	Translation generation and its two-layer cache; <code>visible_videos_qs()</code> and <code>library_videos_qs()</code> which encode the entire visibility model as querysets; <code>user_can_view_video()</code> ; <code>create_share()</code> which preserves the existing token on re-share so links already handed out never break.
services.ai.provider_manager	<code>FallbackAIProvider</code> — a provider that implements the same interface as a real one and delegates with automatic failover. Cached as a module-level singleton via <code>get_ai_provider()</code> .

4.5 Performance considerations found in the code

- **N+1 elimination.** `_with_related()` attaches `select_related("transcript", "summary", "share")` and `prefetch_related("key_moments")` to every visibility queryset, because serialising N videos otherwise issued 3N extra queries.
- **Two serialiser tiers.** `VideoListSerializer` deliberately omits transcript, summary and key-moment payloads; the full `VideoSerializer` is only used for detail views or when `?full=true` is requested.
- **Cheap polling.** `VideoProgressView` returns four scalars, nothing more.
- **Two-layer translation cache.** Redis (7-day TTL) in front of a durable `VideoTranslation` row, so a translation survives restarts and is shared between the web and worker containers.
- **Search cost containment.** Non-matching videos never leave PostgreSQL; segment extraction happens in Python only for videos that already matched, and is capped at 20 matches per video.
- **Streaming uploads.** `DATA_UPLOAD_MAX_MEMORY_SIZE` and `FILE_UPLOAD_MAX_MEMORY_SIZE` are both 10 MB, so a 2 GB video streams to a temporary file instead of being buffered in RAM.

5. Authentication & Role-Based Access Control

5.1 Identity model

`AUTH_USER_MODEL` is a fully custom `accounts.User` built on `AbstractBaseUser` + `PermissionsMixin`. There is no username: `email` is the `USERNAME_FIELD`, unique and indexed. Primary keys are UUID4 rather than sequential integers, so identifiers are non-enumerable. Additional operational fields include `last_login_ip`, `last_login_device`, `is_verified`, `login_provider` and a `deleted_at` column reserved for soft deletion.

5.2 JWT configuration and token lifecycle

Setting	Value	Effect
<code>ACCESS_TOKEN_LIFETIME</code>	60 minutes	Short-lived bearer token sent on every request.
<code>REFRESH_TOKEN_LIFETIME</code>	7 days	Bounds the maximum session length.
<code>ROTATE_REFRESH_TOKENS</code>	True	Every refresh issues a brand-new refresh token.
<code>BLACKLIST_AFTER_ROTATION</code>	True	The consumed refresh token is immediately unusable — a stolen token cannot be replayed after the legitimate client refreshes.
<code>ALGORITHM / SIGNING_KEY</code>	HS256 / <code>JWT_SECRET</code>	Falls back to <code>SECRET_KEY</code> only if <code>JWT_SECRET</code> is blank, so tokens are never signed with an empty key.
<code>UPDATE_LAST_LOGIN</code>	True	Keeps the admin user table's "last login" column meaningful.

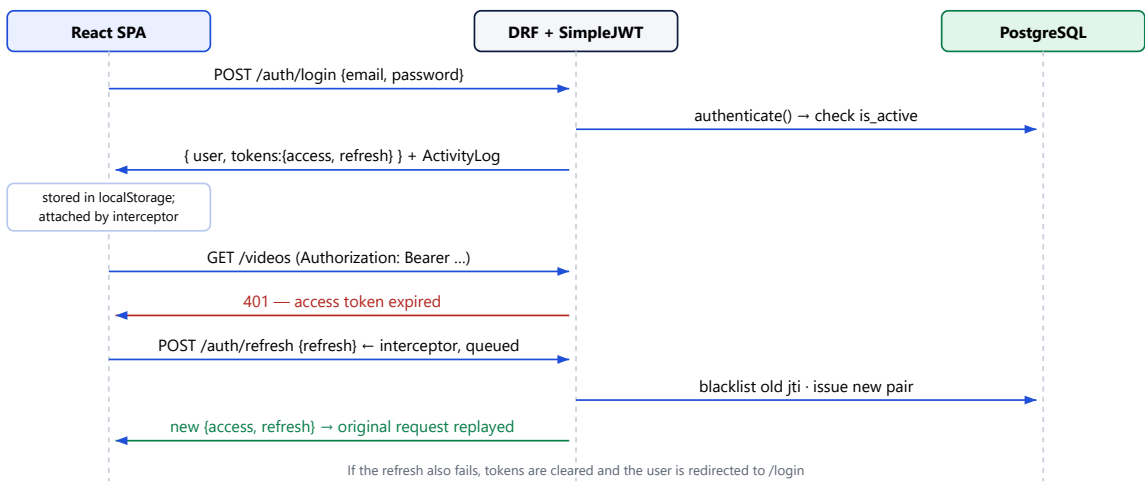


Figure 5.1 — Token lifecycle, including the silent-refresh path implemented in `src/Lib/api.js`.

The client-side refresh implementation is worth calling out: a module-level `isRefreshing` flag and a `failedQueue` array ensure that if ten requests 401 simultaneously, only *one* refresh call is made and the other nine are parked and replayed with the new token. Requests to the public auth endpoints are excluded from both interceptors so a failed login cannot trigger a refresh loop.

5.3 Password and account policy

- Registration enforces a regex policy in `utils/validators.py`: minimum 8 characters with at least one lowercase, one uppercase, one digit and one character from `@$!%*?&`, on top of Django's four standard validators.
- `admin` is absent from `SELF_ASSIGNABLE_ROLES` and re-rejected in `validate_role()` — defence in depth, so the public endpoint can never mint a privileged account.
- Password reset issues a 32-byte URL-safe token stored in Redis for 30 minutes, e-mails a link to `FRONTEND_URL/reset-password?token=...`, and the endpoint returns an identical success message whether or not the address exists, preventing account enumeration.

5.4 Capability matrix

Derived directly from the `permission_classes` on each view.

Capability	Creator	Learner	Educator	Admin	Enforced by
Upload file / submit YouTube URL	yes	no	yes	no	CanUpload
Edit a transcript	own	no	own	no	CanAuthorContent
Create / edit learning materials	yes	no	yes	no	CanAuthorContent
Read learning materials	yes	shared	yes	yes	IsAuthenticated + visibility
Share a video / revoke a share	own	no	own	no	CanAuthorContent + strict owner
View / bookmark shared content	yes	yes	yes	yes	IsAuthenticated
Cross-video transcript search	yes	yes	yes	yes	visible_videos_qs scope
Personal analytics	yes	yes	yes	yes	IsAuthenticated
Content insights	yes	no	yes	no	CanAuthorContent
Classroom analytics	no	no	yes	no	IsEducator
Delete a video	own	no	own	any	owner-or-admin queryset
User & role management, settings, audit	no	no	no	yes	IsAdmin

404, not 403, on unauthorised reads. `_get_viewable_video()` returns *"Video not found."* both when a video does not exist and when the caller may not see it. The API therefore never confirms that a given UUID exists, closing an object-enumeration side channel.

6. Database Design

PostgreSQL holds 27 tables, of which 16 belong to the application (the remaining 11 are Django auth, contenttypes, sessions, admin log and SimpleJWT blacklist tables). Every application model uses a UUID primary key and an explicit `db_table` name.

6.1 Entity-relationship diagram

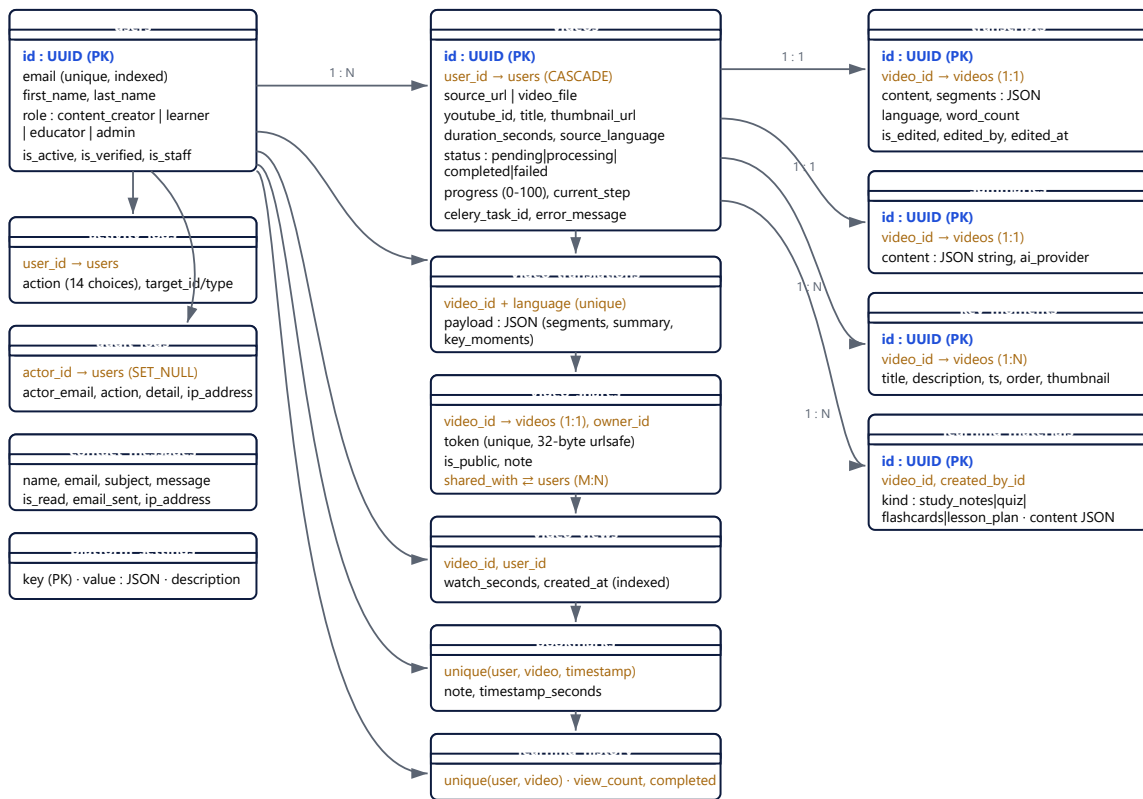


Figure 6.1 — Entity-relationship diagram. Blue = primary key, amber = foreign key. Every FK from `videos` cascades, so deleting a video removes its transcript, summary, key moments, translations, share, views, bookmarks and history in one transaction.

6.2 Table reference

Table	App	Purpose and notable constraints
users	accounts	Custom user. UUID PK, unique indexed e-mail, four-value role choice, ordering by <code>-created_at</code> .
activity_logs	accounts	User-facing history, 14 action choices. Composite index on <code>(user, -created_at)</code> . <code>target_id</code> is a loose <code>CharField</code> , not an FK, so history survives deletion of its target.
audit_logs	accounts	Privileged-action trail. <code>actor</code> is <code>SET_NULL</code> and <code>actor_email</code> is denormalised, so the trail outlives the administrator's account.
contact_messages	accounts	Public enquiries. Persisted <i>before</i> mail is attempted; <code>email_sent=False</code> flags an SMTP outage rather than losing the message.
platform_settings	accounts	Key/value JSON configuration with the key as the primary key.
videos	videos	The pipeline's state machine: status, 0–100 progress, human-readable step, Celery task id and error text all live on this row, which is why the admin job monitor needs no separate job table.
video_translations	videos	<code>unique_together (video, language)</code> . Durable translation store beneath the Redis cache.
video_shares	videos	<code>OneToOne</code> with the video; unique indexed token. A single row carries both a public link and an optional recipient list.
video_shares_shared_with	videos	M:N join between a share and its named recipients. Empty = visible to every authenticated user.
video_views	videos	Raw playback events; two composite indexes for the analytics aggregations.
bookmarks	videos	<code>unique_together (user, video, timestamp_seconds)</code> , making re-bookmarking idempotent while still allowing several distinct timestamped bookmarks per video.
learning_history	videos	<code>unique_together (user, video)</code> , updated in place so history reads as a most-recently-viewed list rather than an event stream.
transcripts	transcripts	Plain text plus a JSON <code>segments</code> array of <code>{start, duration, text}</code> . Human-correction provenance via <code>is_edited / edited_by / edited_at</code> .
summaries	summaries	The eight-section analysis serialised as a JSON string in a <code>TextField</code> , plus the provider name that produced it.
key_moments	summaries	Ordered highlights with a rendered <code>ImageField</code> thumbnail.
learning_materials	summaries	Educator artefacts; the JSON <code>content</code> shape depends on <code>kind</code> .

6.3 Migrations

Sixteen application migrations are applied, all reversible Django migrations under version control: five for `accounts`, six for `videos`, two for `transcripts` and three for `summaries`. Two of them are notable because they are not simple schema additions:

- `accounts/0004_migrate_legacy_user_role` — a **data** migration that rewrites existing rows onto the current four-role vocabulary.
- `videos/0002_drop_clips_table` — removes an abandoned `clips` concept, showing the schema was actively pruned rather than left to accumulate dead tables.

6.4 Storage of AI output

The eight summary sections are stored as one JSON document rather than eight columns. The reason is visible in the code path: the pipeline writes `json.dumps(summary_sections)` in one operation, and both `SummarySerializer.get_content()` and `LearningMaterialListView._build_from_summary()` read it back with `json.loads` and a defensive fallback that treats a non-JSON body as `{"detailed_summary": ...}`. This keeps the LLM's output schema free to evolve without a migration, at the cost of not being able to query inside a summary at the SQL level. Content insights therefore aggregate topics in Python after loading the JSON.

7. The AI Processing Pipeline

The pipeline is a single bound Celery task, `process_video_task` in `apps/processing/tasks.py`. It is dispatched with `.delay(str(video.id))` the moment a video row is created, so the HTTP request returns `201 Created` immediately and the user is redirected to the live progress screen.

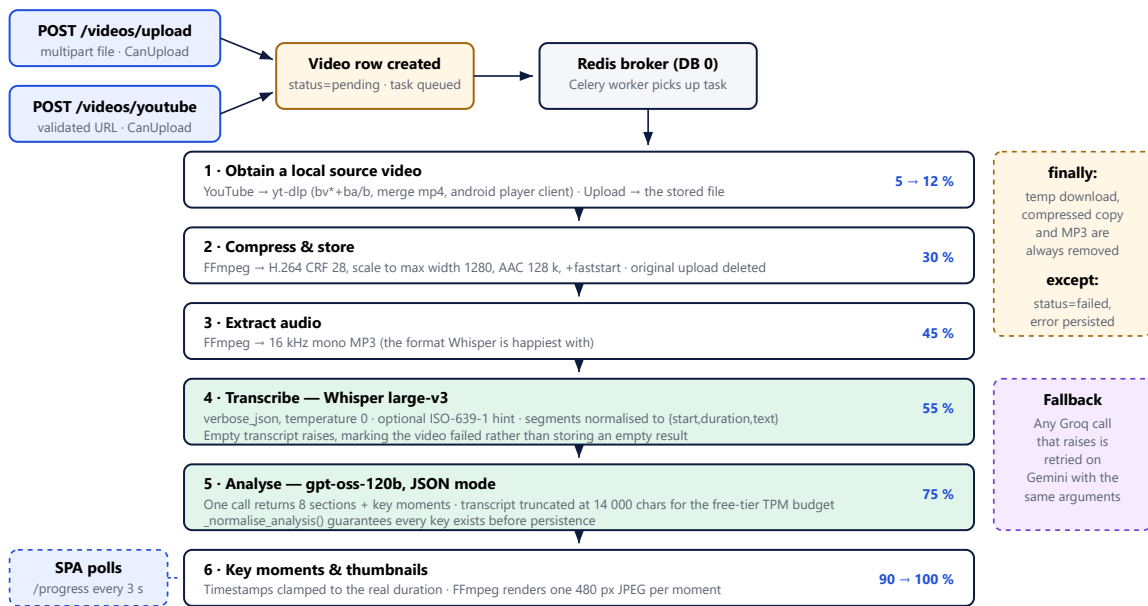


Figure 7.1 — The six pipeline stages with their exact progress checkpoints, cleanup guarantees and failover behaviour.

7.1 Provider abstraction

`services/ai/base.py` defines `BaseAIProvider`, an ABC with four abstract methods: `transcribe()`, `summarize()`, `analyze_transcript()` and `translate_json()`, plus a `provider_name` property. Two concrete providers implement it. The interesting piece is that the resilience layer is *itself* a provider: `FallbackAIProvider` implements the same interface, so the pipeline calls it without knowing that failover exists.

```

def transcribe(self, audio_file_path, language=None):
    """Try Groq STT first, fall back to Gemini."""
    if self._groq_provider:
        try:
            return self._groq_provider.transcribe(audio_file_path, language=language)
        except Exception as exc:
            logger.warning("Groq STT failed; switching to Gemini fallback. Error: %s", exc)
            if self._gemini_provider:
                return self._gemini_provider.transcribe(audio_file_path, language=language)
            raise exc
    # ... otherwise Gemini alone, or AIProviderError if neither key is configured.
  
```

Operation	Primary (Groq)	Fallback (Gemini)	Notes
Speech to text	whisper-large-v3	Gemini Files API + <code>gemini-flash-latest</code>	Groq returns real segment timings; the Gemini path asks for the same JSON shape in the prompt.
Transcript analysis	openai/gpt-oss-120b	<code>gemini-flash-latest</code>	Both use the identical prompt constant, imported from <code>groq_provider</code> .
Translation	openai/gpt-oss-120b	<code>gemini-flash-latest</code>	JSON-object response mode; keys, numbers and timestamps must be preserved.

7.2 Prompt design

A single prompt constant produces the entire analysis in one JSON-mode call. Beyond the schema it carries four explicit rules that address failure modes the authors evidently hit in practice: never invent facts the transcript does not support; keep every `timestamp_seconds` inside the real time range; write explanations in English even when the audio is not; and still populate the sections for sparse-speech content such as songs. Because the model can still return an out-of-range timestamp, the pipeline clamps it at persistence time and logs a warning — belt *and* braces.

7.3 On-demand translation

Translation is a separate, user-triggered flow (`POST /videos/<id>/translate`) rather than part of the pipeline. It never modifies the original records. Transcript segments are translated in batches of 40 to stay inside the free-tier token budget, and only the `text` field of each segment is replaced — `start` and `duration` are copied through verbatim so the player stays in sync. Summary sections and key-moment text travel in one further combined call. If a batch comes back with the wrong number of items, the original text is used for that batch rather than corrupting alignment.

The result is cached twice: written to a `VideoTranslation` row and to Redis for seven days. Thumbnails and key-moment ids are deliberately *not* stored in the cached payload — they are re-attached on every response — which keeps the stored translation host-independent. Editing a transcript deletes all of that video's translations, so

the next request regenerates from the corrected text.

8. REST API Reference

All endpoints are versioned under `/api/v1/`. The running server publishes an OpenAPI 3.0 schema at `/api/schema/` containing **40 documented paths**, browsable through Swagger UI at `/api/docs/` and ReDoc at `/api/redoc/`.

8.1 Response envelope

SUCCESS

```
{
  "success": true,
  "message": "Videos retrieved.",
  "data": { ... }    // or [ ... ]
}
```

FAILURE

```
{
  "success": false,
  "message": "Registration failed.",
  "errors": { "email": ["Email already registered."] }
}
```

Status	When it is returned
200	Successful read or mutation.
201	Registration, upload/YouTube submission, learning-material creation, contact submission.
400	Validation failure or a malformed request body.
401	Missing/expired token, or bad credentials at login.
403	Authenticated but the role does not permit the action.
404	Not found or not visible to the caller — the API does not distinguish.
429	Throttled (100/day anonymous, 1000/day authenticated, 5/hour on the contact form).
502	All AI providers failed (translation, analysis).

8.2 Authentication — `/api/v1/auth/`

Method	Path	Access	Purpose
POST	<code>/register</code>	Public	Create an account. Role limited to creator/learner/educator.
POST	<code>/login</code>	Public	Returns the profile and an access/refresh pair; writes a <code>login</code> activity entry.
POST	<code>/refresh</code>	Public	Rotates the pair and blacklists the consumed refresh token.
POST	<code>/logout</code>	Authenticated	Blacklists the supplied refresh token.
POST	<code>/forgot-password</code>	Public	Issues a 30-minute reset token and mails the link. Always reports success.
POST	<code>/reset-password</code>	Public	Consumes the token and sets the new password.
POST	<code>/change-password</code>	Authenticated	Requires the current password.
GET / PATCH	<code>/profile</code>	Authenticated	Read or update name and profile image (multipart accepted).
GET	<code>/activity</code>	Authenticated	Paginated personal activity history, filterable by <code>?action=</code> .

8.3 Videos & content — `/api/v1/videos/`

Method	Path	Access	Purpose
POST	<code>/upload</code>	<code>CanUpload</code>	Multipart upload; validates extension, size and container magic bytes, then queues the pipeline.
POST	<code>/youtube</code>	<code>CanUpload</code>	Submits a validated YouTube URL with an optional language hint.
GET	<code>/</code>	Authenticated	Visible videos. Supports <code>?search=</code> , <code>?status=</code> , <code>?full=true</code> , <code>?paginate=true</code> .
GET	<code>/library</code>	Authenticated	Completed videos shared with the caller, excluding their own — the Learner's browse surface.
GET	<code>/search?q=</code>	Authenticated	Spoken-word search with timestamped matches, scoped to visible videos.
GET / DELETE	<code>/<id></code>	Viewer / owner or admin	Full detail, or delete the video and its media from disk.
GET	<code>/<id>/progress</code>	Viewer	Four-field polling endpoint.
POST	<code>/<id>/translate</code>	Viewer	Translated transcript, summary and key moments in the requested language.
PATCH	<code>/<id>/transcript</code>	<code>CanAuthorContent</code>	Correct the transcript; timings preserved, translations invalidated, provenance recorded.
GET/POST/DELETE	<code>/<id>/share</code>	<code>CanAuthorContent</code> + owner	Inspect, create/update or revoke a share. Re-sharing keeps the existing token.
GET	<code>/shared/<token></code>	<code>Public</code>	Unauthenticated share page payload; presentation fields only.
POST	<code>/<id>/view</code>	Viewer	Records a view event and upserts learning history; ≥90 % watched marks completion.
GET/POST	<code>/bookmarks</code>	Authenticated	List or add a bookmark, optionally pinned to a timestamp.
DELETE	<code>/bookmarks/<id></code>	Owner	Remove a bookmark.
GET/DELETE	<code>/history</code>	Authenticated	Read or clear the caller's learning history.
GET/POST	<code>/materials</code>	Read: any · Write: <code>CanAuthorContent</code>	List materials for visible videos, or generate one from the stored analysis.
GET/PATCH/DELETE	<code>/materials/<id></code>	Creator	Read, edit or delete an owned material.

8.4 Analytics, administration and public endpoints

Method	Path	Access	Returns
GET	/analytics/	Authenticated	Personal metrics: counts by status, words transcribed, storage, durations, bookmark/history counters and a 14-day dense upload series.
GET	/analytics/content	CanAuthorContent	Per-video views, unique viewers, bookmarks, key-moment counts, plus a top-20 topic cloud aggregated from AI summaries.
GET	/analytics/classroom	IsEducator	Student views, unique students, completions, average watch seconds, a per-student engagement table and a 14-day series. The educator's own views are excluded.
GET	/admin/stats	IsAdmin	Platform totals: users by role, videos by status, transcripts, summaries, views, bookmarks, storage, 7-day growth.
GET	/admin/users	IsAdmin	All accounts with per-user video count and storage; searchable and role-filterable.
PATCH/DELETE	/admin/users/<id>	IsAdmin	Change role or active status, or delete. Self-demotion and self-deactivation are blocked; both write audit entries.
GET	/admin/activity	IsAdmin	Paginated platform-wide activity feed.
GET	/admin/content	IsAdmin	Every uploaded video for moderation.
GET	/admin/jobs	IsAdmin	Live pipeline monitor: active, queued, failed, completed, 24-hour success rate and recent failures.
GET	/admin/storage	IsAdmin	Totals by user and by container format, media-root and thumbnail footprint.
GET	/admin/audit-logs	IsAdmin	Privileged-action trail.
GET/PATCH	/admin/settings	IsAdmin	Five typed settings; unknown keys and type changes are rejected, and every change is audited.
GET/PATCH	/admin/contact-messages	IsAdmin	Read and triage public enquiries.
POST	/contact	Public	Contact form; 5 requests/hour per IP, persisted before mail is attempted.
GET	/health/	Public	Database and cache probe for the container health check.

8.5 Worked examples

Submitting a YouTube video

```
POST /api/v1/videos/youtube
Authorization: Bearer <access token>
Content-Type: application/json

{ "url": "https://www.youtube.com/watch?v=0oumFBCLw3s", "language": "hi" }

--- 201 Created ---
{
  "success": true,
  "message": "YouTube analysis started.",
  "data": {
    "id": "d7c1830a-32c2-403a-9b71-52cad5a84836",
    "title": "YouTube Video",           // replaced with the real title during ingestion
    "status": "pending", "progress": 0, "current_step": "Pending",
    "source_language": "hi", "transcript": null, "summary": null, "key_moments": []
  }
}
```

Polling progress

```
GET /api/v1/videos/d7c1830a-.../progress

{ "success": true, "message": "Progress retrieved.",
  "data": { "status": "processing", "progress": 55,
    "current_step": "Generating Transcript", "error_message": "" } }
```

Cross-video spoken-word search

```
GET /api/v1/videos/search?q=बाबा

{ "success": true, "message": "Search complete.",
  "data": {
    "query": "बाबा", "video_count": 1, "total_matches": 2,
    "results": [{
      "video_id": "d7c1830a-...",
      "title": "We can't live just for humans, we have to serve back to someone.",
      "owner_name": "educator nikul", "duration_seconds": 178.9, "is_owner": true,
      "match_count": 2,
      "matches": [ { "start": 0.0, "text": "बाबा जी आ रहे थे उपर से बरफ में से नंगे पैर" },
        { "start": 7.0, "text": "तो बाबा जी ने थोड़े ज़र के लिए कंप्यूज़ हो गए" } ]
    } ]
  } }
```

The structured summary payload

`SummarySerializer` parses the stored JSON string before returning it, so the client receives an object, not an escaped string:

```
GET /api/v1/videos/<id> → data.summary

{ "id": "...", "word_count": 234, "ai_provider": "Resilient Groq (with Gemini Fallback)",
  "content": {
    "short_summary": "...2-3 sentence overview...",
    "detailed_summary": "...multi-paragraph prose...",
    "bullet_summary": ["6-12 dense takeaways"],
    "chapter_summary": [{ "chapter": "...", "summary": "...", "timestamp_seconds": 0.0 }],
    "important_topics": [...], "action_items": [...],
    "glossary": [{ "term": "...", "definition": "..."}],
    "key_questions": [{ "question": "...", "answer": "..."}]
  }
}
```

Sharing a video

```
POST /api/v1/videos/<id>/share
{ "is_public": true, "note": "Week 4 lecture",
  "emails": ["student1@example.com"] } // omit "emails" to share with all learners

{ "success": true, "message": "Video shared.",
  "data": { "token": "kQ8f...32-byte-unsafe", "is_public": true,
    "share_url": "http://localhost:3000/shared/kQ8f...", "recipient_count": 1 } }
```

Only completed videos can be shared, and the share endpoint uses *strict* ownership rather than the owner-or-admin rule used for deletion — publishing somebody else's video is treated as an authoring act, not a moderation act.

8.6 Media delivery

Video files are private. Because a browser `<video>` element cannot attach an `Authorization` header, serialisers hand out URLs of the form `/media/videos/<uuid>.mp4?t=<signed-token>`. The token is a `TimestampSigner` signature over that exact relative path with a 12-hour lifetime, so it cannot be replayed against a different file. `serve_media` then authorises through three independent routes — a valid signed token, a public share, or an authenticated owner/admin/recipient — normalises the path to block traversal outside `MEDIA_ROOT`, and answers `Range` requests with `206 Partial Content` so seeking works. Thumbnails and profile images are treated as non-sensitive derivatives and are not signed.

9. Frontend Implementation

The SPA is roughly 6,800 lines of JSX across 23 pages, 16 components, 2 context providers, 2 route guards and 5 library modules. There is no Redux, no data-fetching library and no component library: state is two React contexts, styling is Tailwind utility classes, and every server call goes through one configured Axios instance.

9.1 Structure

```
clip_mind_AI/src/
├─ main.jsx           # AuthProvider → VideoProvider → App
├─ App.jsx           # all routes; public vs. protected vs. role-gated
├─ context/
│  ├─ AuthContext.jsx # user, loading, login/register/logout/reset/change
│  └─ VideoContext.jsx # videos, activeVideo, polling, translation, sharing,
│                       # bookmarks, view tracking, transcript editing
├─ routes/
│  ├─ ProtectedRoute.jsx # requires a session
│  └─ RoleRoute.jsx      # requires one of `allow`; redirects to homeForRole(role)
├─ layouts/DashboardLayout.jsx # Sidebar + TopNavbar + <Outlet/>
├─ lib/
│  ├─ api.js           # Axios instance, JWT attach, single-flight refresh queue
│  ├─ roles.js         # ROLES, UPLOADER_ROLES, AUTHOR_ROLES, nav, homeForRole()
│  ├─ player.js        # seekAndPlay(): metadata-aware seek then play
│  └─ motion.js site.js
├─ components/
│  ├─ auth/ (5)
│  ├─ home/ (5)
│  ├─ dashboard/ (2)
│  └─ ui/Toast.jsx Navbar Footer
└─ pages/ (23)
```

9.2 Routing and guards

Route	Guard	Page
/ · /features · /about · /contact	public	Marketing site — hero, feature grid, role showcase, how-it-works, FAQ, contact form.
/login · /register · /forgot-password · /reset-password	public	Authentication screens sharing AuthLayout .
/shared/:token	public	SharedVideo — deliberately outside ProtectedRoute .
/dashboard · /transcript · /summary · /key-moments · /search · /bookmarks · /history · /library · /learning-materials	content roles	The main working surfaces, inside DashboardLayout .
/upload · /processing	UPLOADER_ROLES	Ingestion and the live pipeline view.
/analytics	AUTHOR_ROLES	Content insights.
/classroom	educator only	Classroom analytics.
/admin	admin only	Eight-tab administrator console.

homeForRole() deserves a mention because it fixes a real class of bug: an administrator bounced to /dashboard (which they also cannot access) would redirect forever, so every guard and post-login redirect routes through a single function that returns /admin for administrators and /dashboard for everyone else.

9.3 State management

Context	Responsibilities
AuthContext	Restores the session on mount by calling /auth/profile with the stored token (letting the Axios interceptor silently refresh if needed), exposes login , register , logout , forgotPassword , resetPassword and changePassword , and holds a loading flag that guards render until the restore completes.
VideoContext	Owns the video list and the "active video"; polls progress on a 3-second interval only while processing; fetches, caches and exposes translations; provides upload, YouTube submission, delete, share/unshare, bookmark, view-tracking and transcript-update actions; and derives a single display object that transparently returns either the original or the translated transcript, summary and key moments so pages never branch on language.

9.4 Notable client-side implementation details

- Single-flight token refresh.** Concurrent 401s queue behind one refresh call (isRefreshing + failedQueue) and are then replayed with the new token.
- Reliable seeking.** seekAndPlay() waits for readyState ≥ HAVE_METADATA , sets currentTime , waits for the seeked event, and only then calls play() — which is what makes "jump to key moment" land on the right frame instead of restarting at 0:00.
- Engagement tracking that cannot break playback.** recordView is fire-and-forget; a failed analytics ping is logged at debug level and swallowed. The furthest position reached is reported once on unmount rather than streaming a request per second.
- Editing guards against a real hazard.** Transcript editing always drafts from activeVideo.transcript.segments — the original — never from the translated view, so saving while a translation is displayed cannot overwrite the source text.

- **Owner-aware dashboard.** Because `GET /videos` also returns content shared *with* the user, the dashboard filters on `is_owner` for uploaders so "Your Videos" cannot contradict the stat cards, which count owned videos only.
- **Persisted UI state.** The selected video and the display language survive reloads via `localStorage`.

10. Application Walkthrough

Every image in this section is a screenshot of the running application, captured from the Docker Compose stack at `http://localhost:3000` against a live PostgreSQL database containing real processed videos, AI-generated summaries and engagement records.

10.1 Public site and onboarding

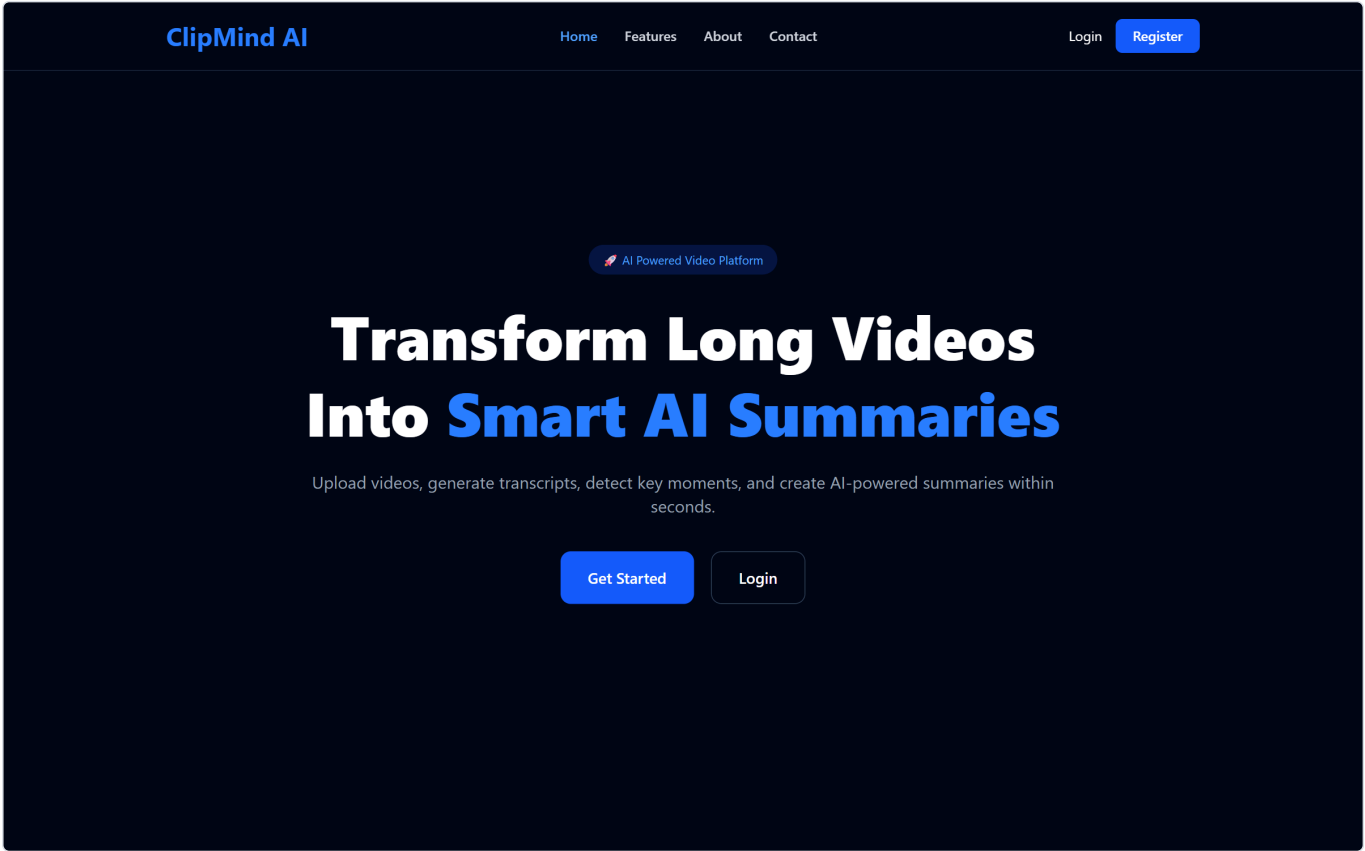


Figure 10.1 — Landing page. Public routes are rendered without the dashboard shell.

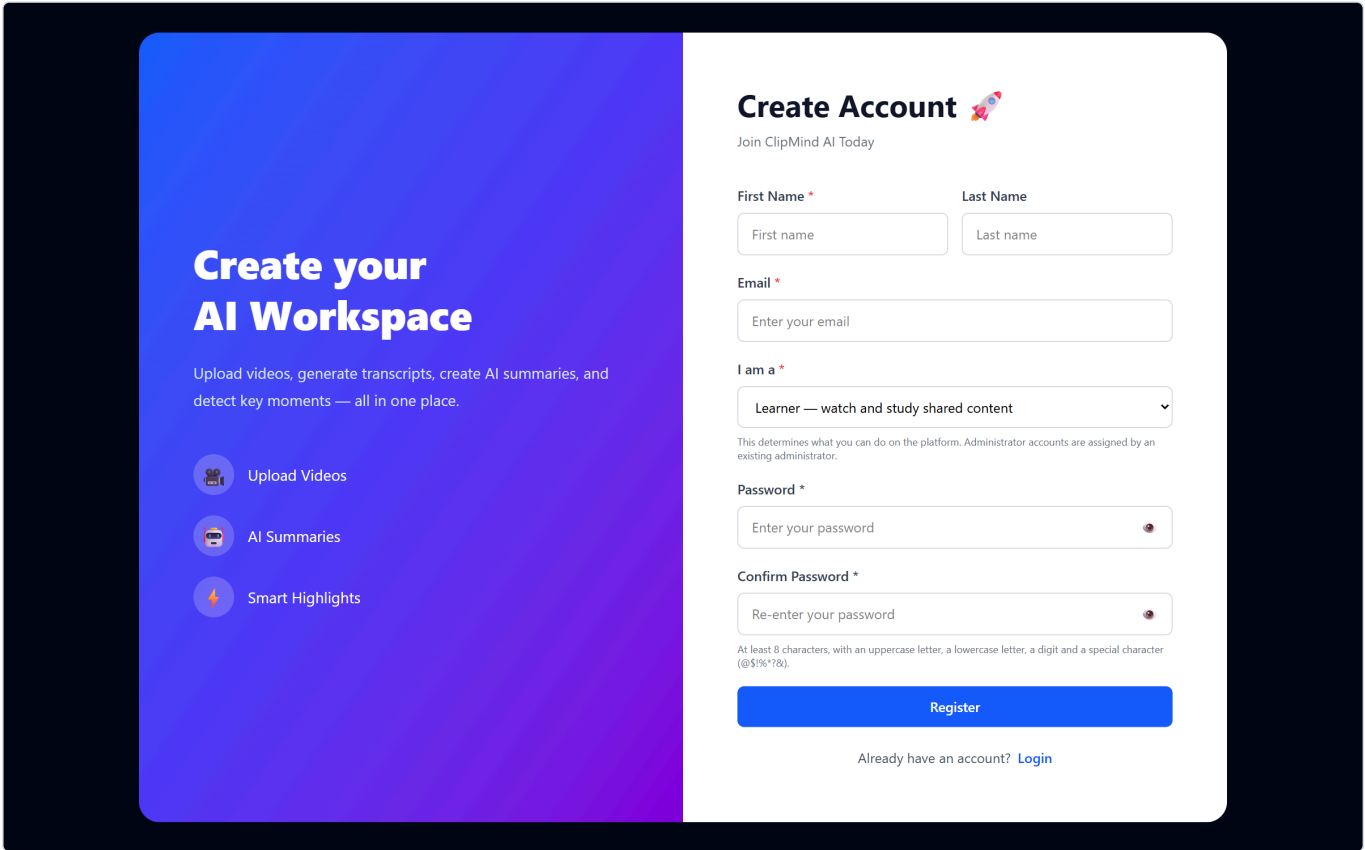


Figure 10.2 — Registration. The role selector offers only Content Creator, Learner and Educator; the administrator role is rejected server-side even if the payload is tampered with.

10.2 Ingestion and the AI pipeline

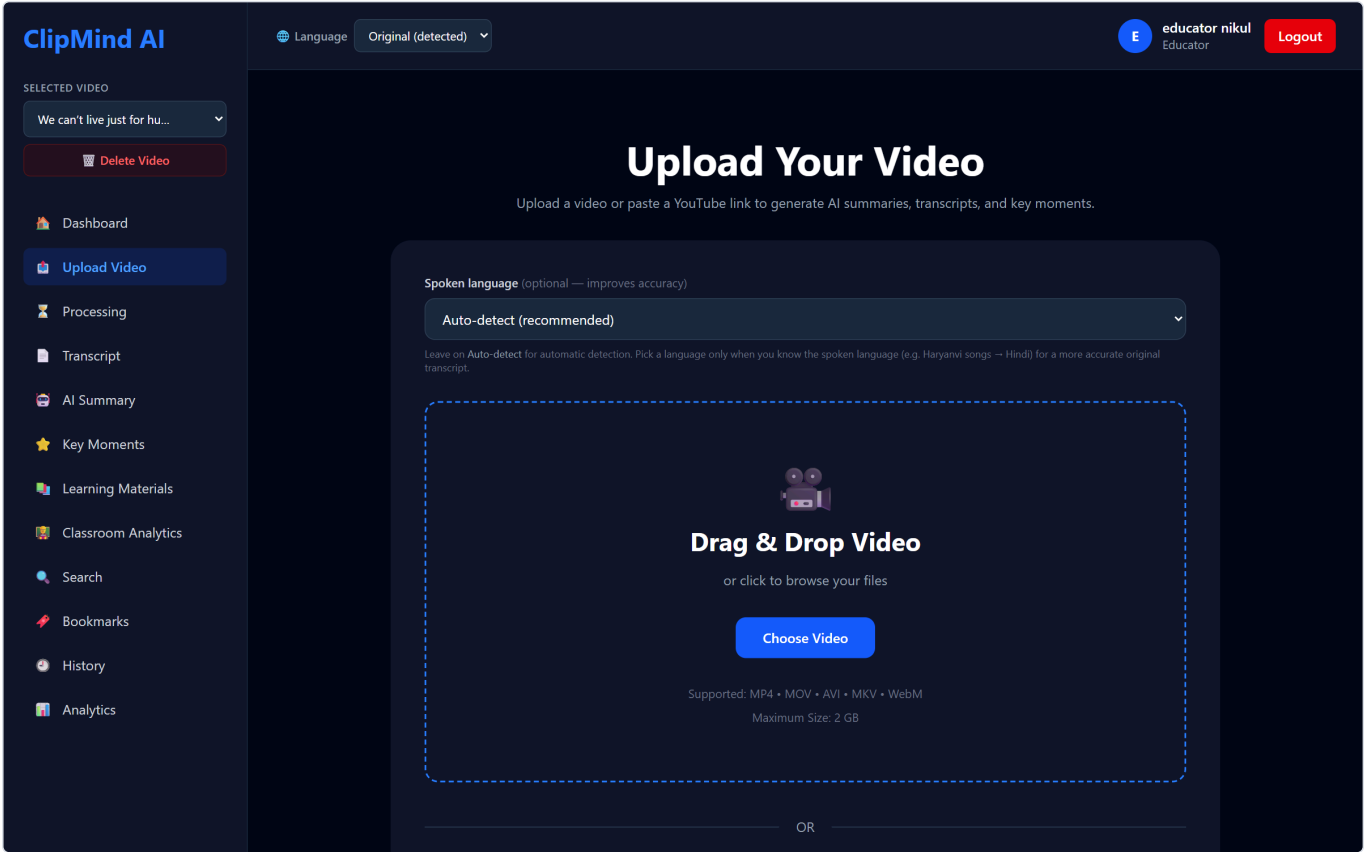


Figure 10.3 — Dual ingestion. Drag-and-drop or file picker for local media, a YouTube URL field below, and an optional spoken-language hint that is validated against the supported ISO-639-1 set before reaching the pipeline.

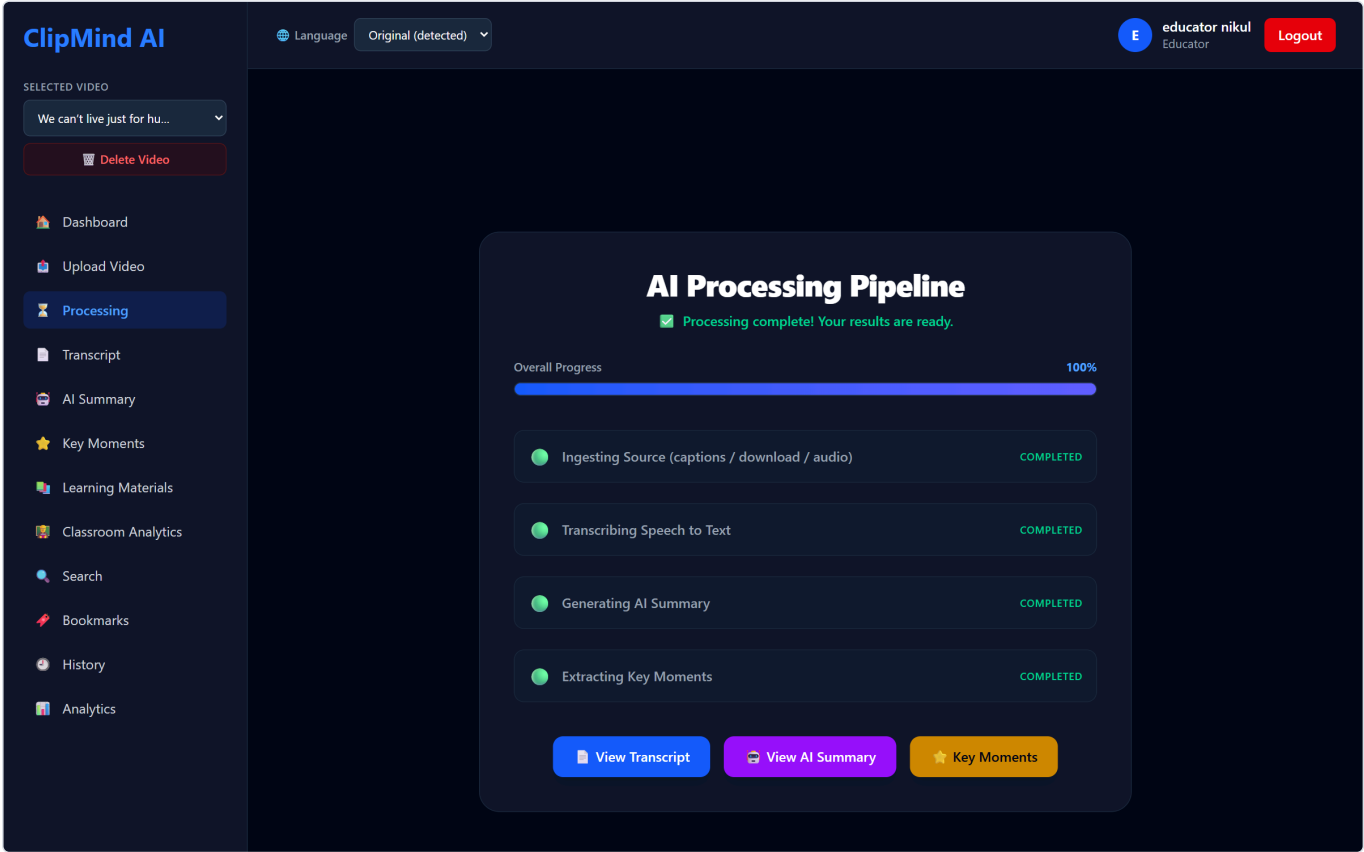


Figure 10.4 — The live pipeline view. Stage states are derived from the numeric `progress` value returned by the polling endpoint, so the UI stays correct regardless of which ingestion branch ran.

10.3 Transcript and summary

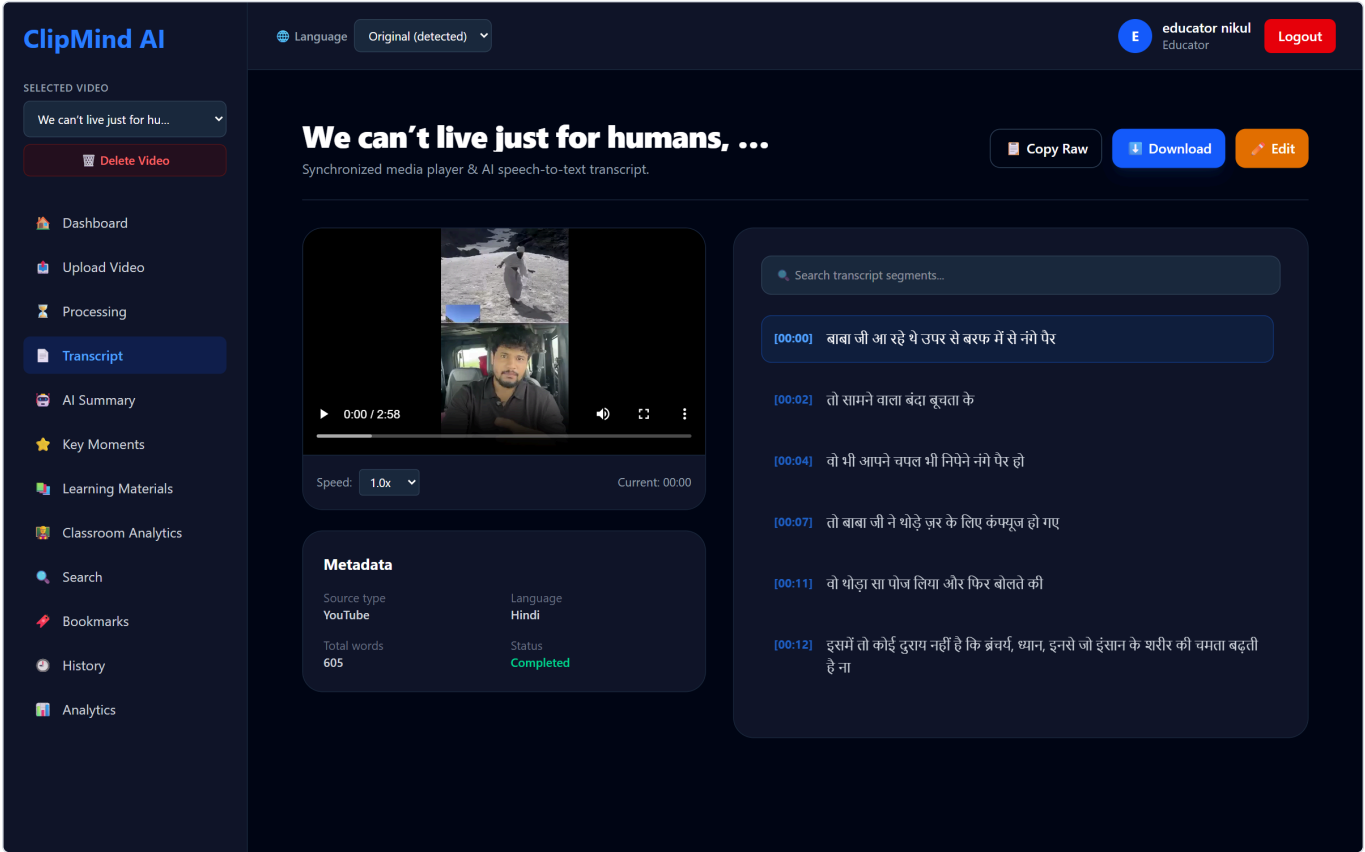


Figure 10.5 — Synchronised transcript. The player is fed by a signed media URL; the active segment is highlighted from `timeupdate`, clicking a timestamp seeks the player, and the metadata panel reports the auto-detected language (Hindi here) and word count. The Edit action is visible because this user is an Educator and the video's owner.

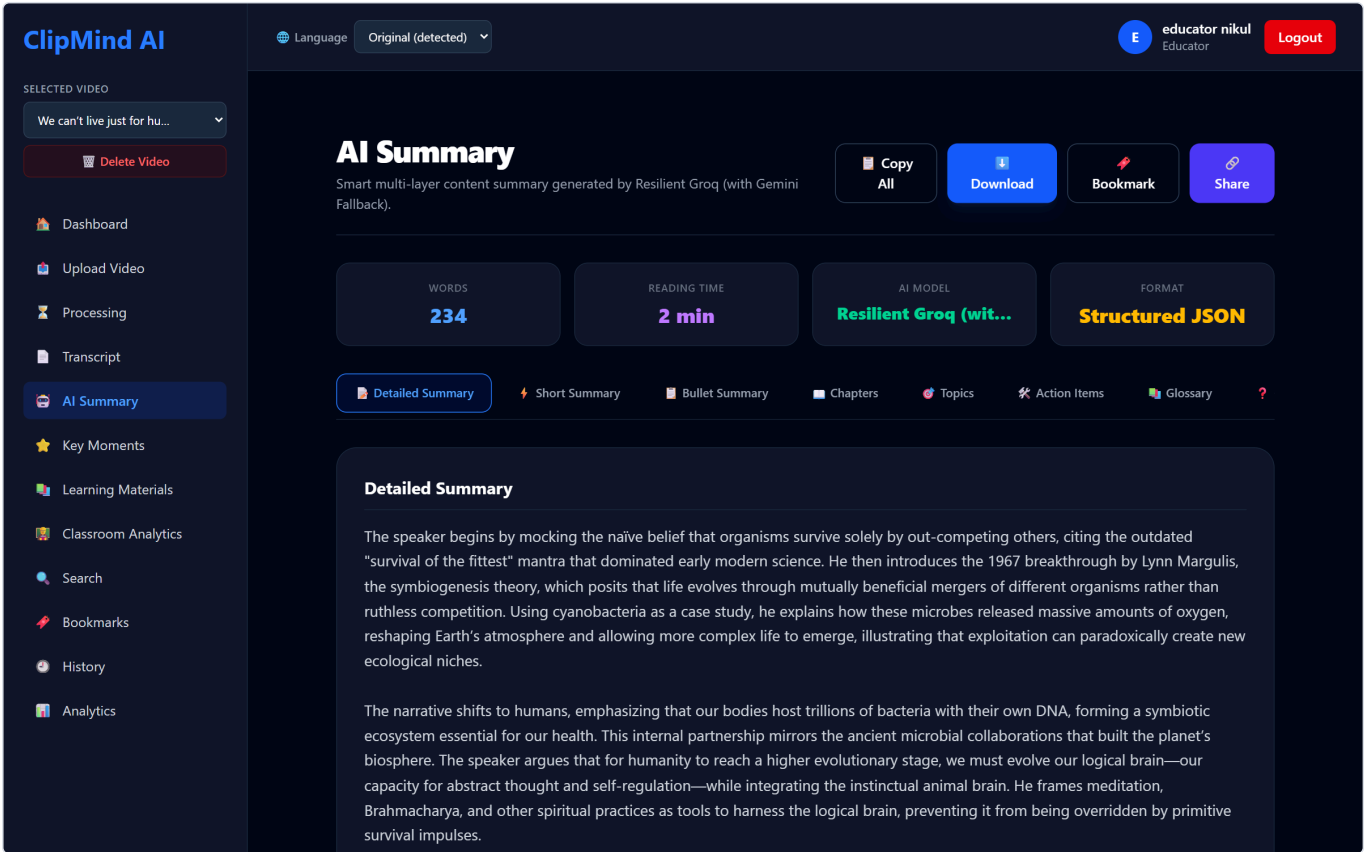


Figure 10.6 — The eight-section AI summary. The header reports the provider that actually produced it — "Resilient Groq (with Gemini Fallback)" — and the tab strip maps one-to-one onto the JSON keys returned by the analysis call.

10.4 Key moments and search

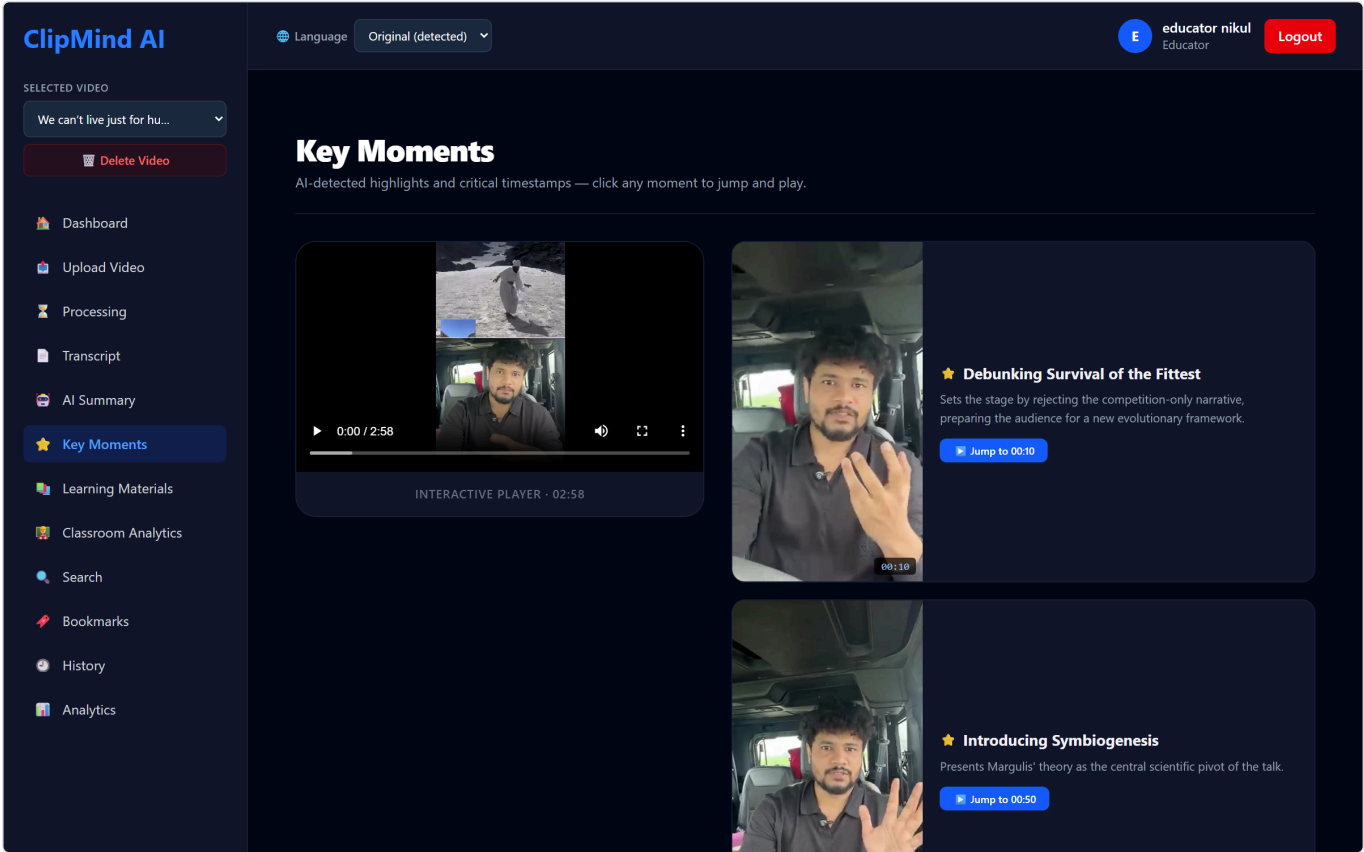


Figure 10.7 — Key moments. Each card carries an AI-written title and rationale, a timestamp and a thumbnail that FFmpeg rendered from that exact frame during processing. "Jump to" seeks the embedded player.

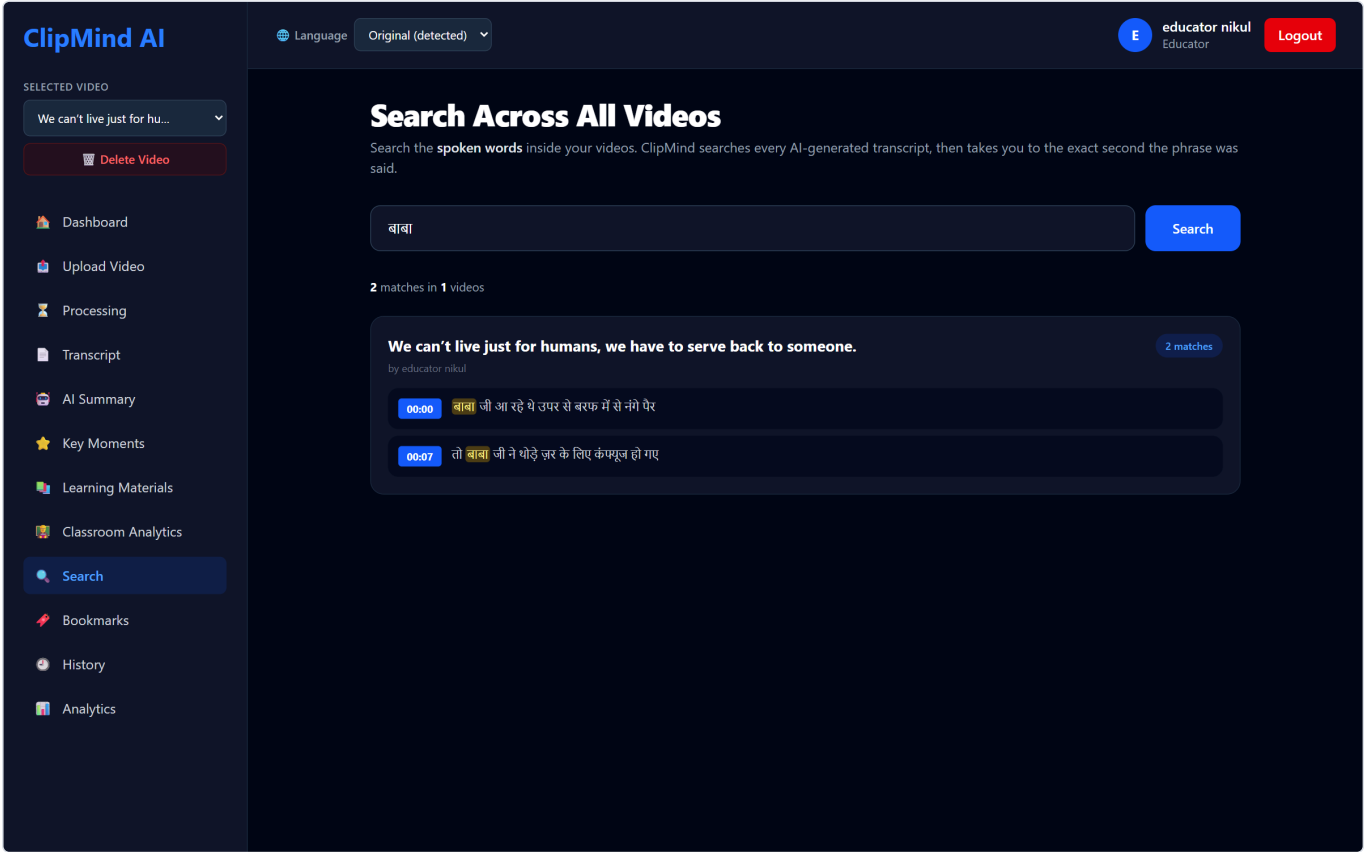


Figure 10.8 — Spoken-word search. The query runs against transcript content in PostgreSQL, matching segments are extracted with their start times, and the matched term is highlighted. Results are scoped to videos the caller may see.

10.5 Educator surfaces

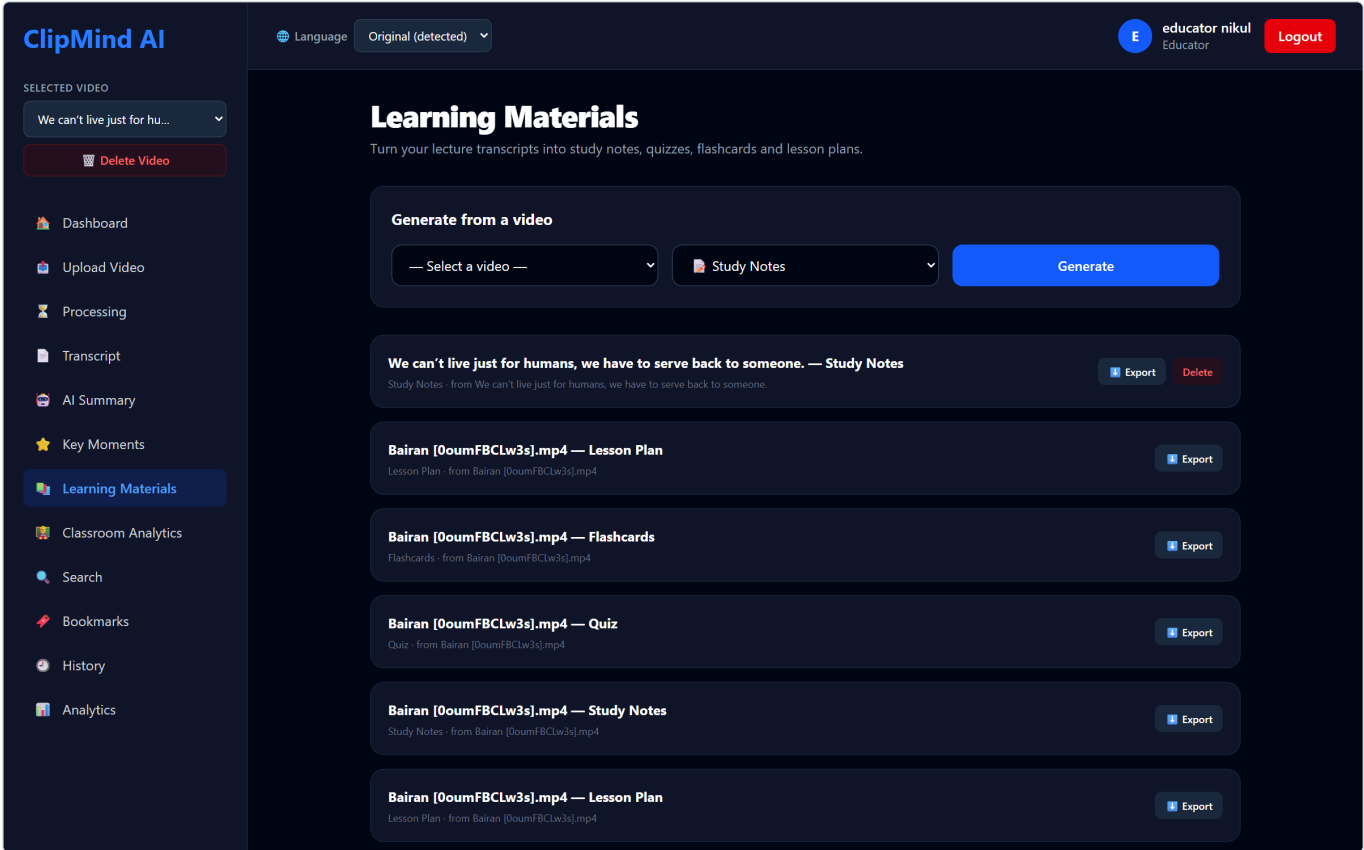


Figure 10.9 — Learning materials. Study notes, quizzes, flashcards and lesson plans are assembled from the analysis the pipeline already produced, so generating one costs no additional LLM call. Each renders according to its `kind` and exports as JSON.

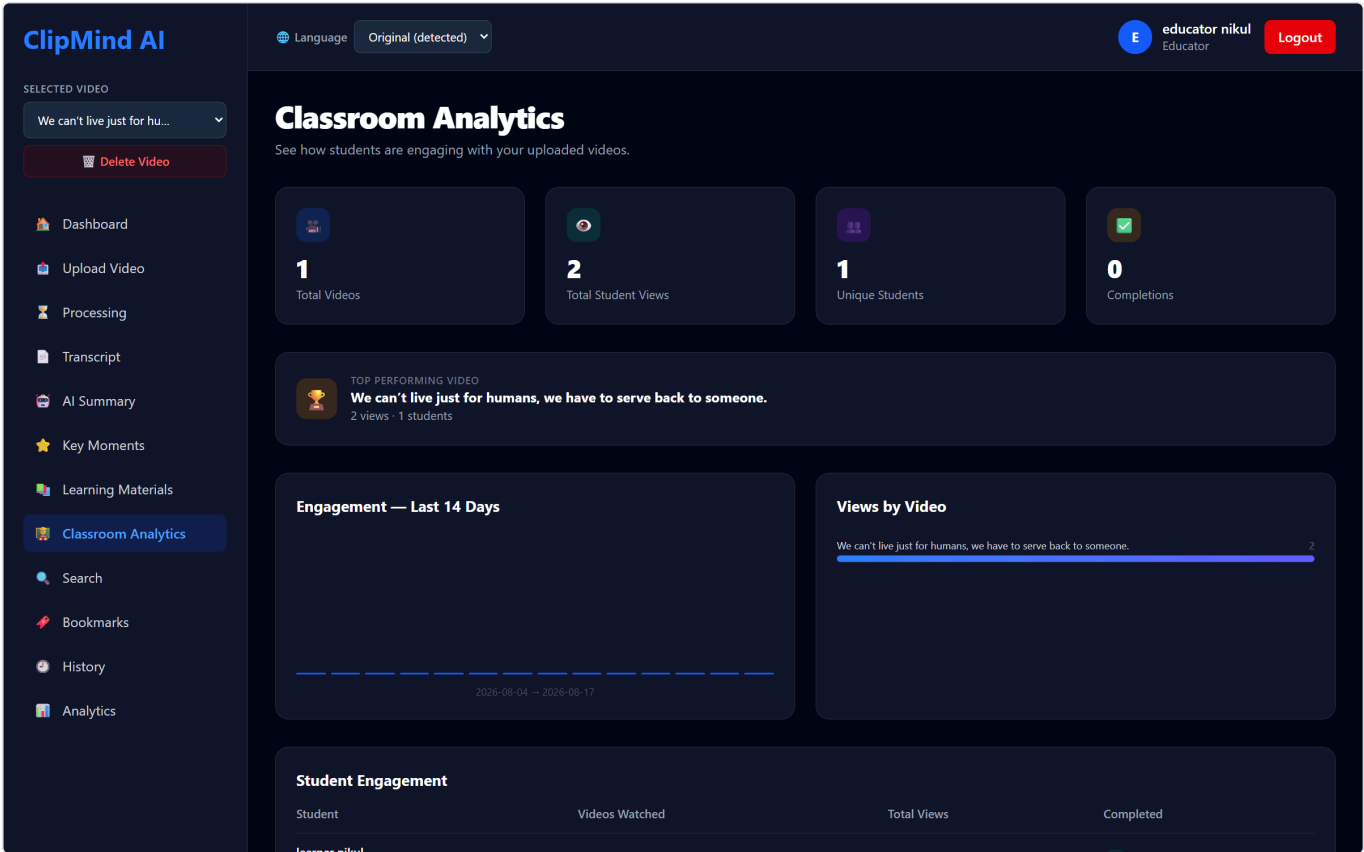


Figure 10.10 — Classroom analytics. Views, unique students, completions and average watch time are aggregated from `video_views` and `Learning_history`, with the educator's own views excluded so self-review does not inflate the numbers.

10.6 Learner surfaces

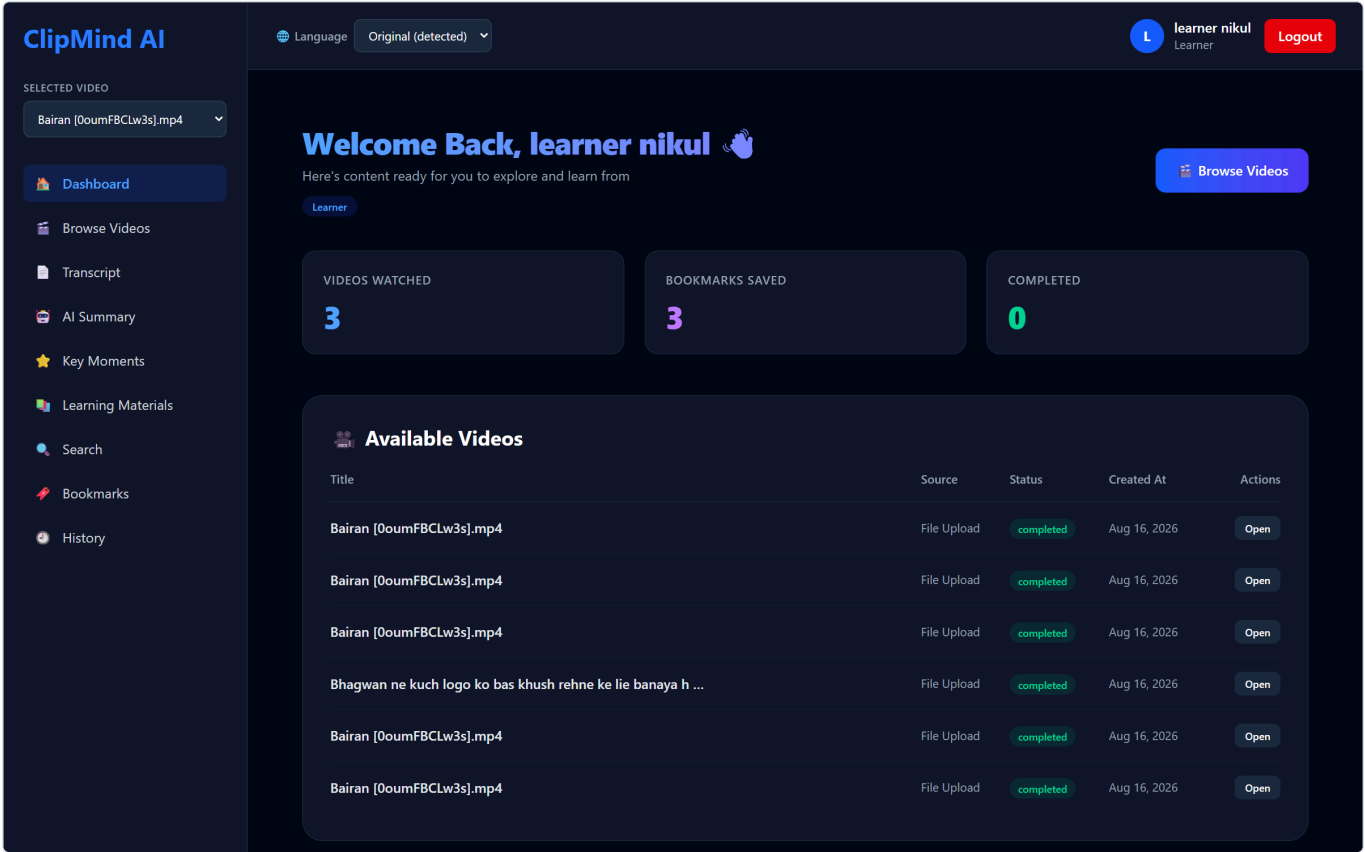


Figure 10.11 — The Learner dashboard. The same component renders consumption metrics instead of production metrics, and the primary call to action becomes "Browse Videos" rather than "Upload Video". The sidebar contains no upload or processing entries.

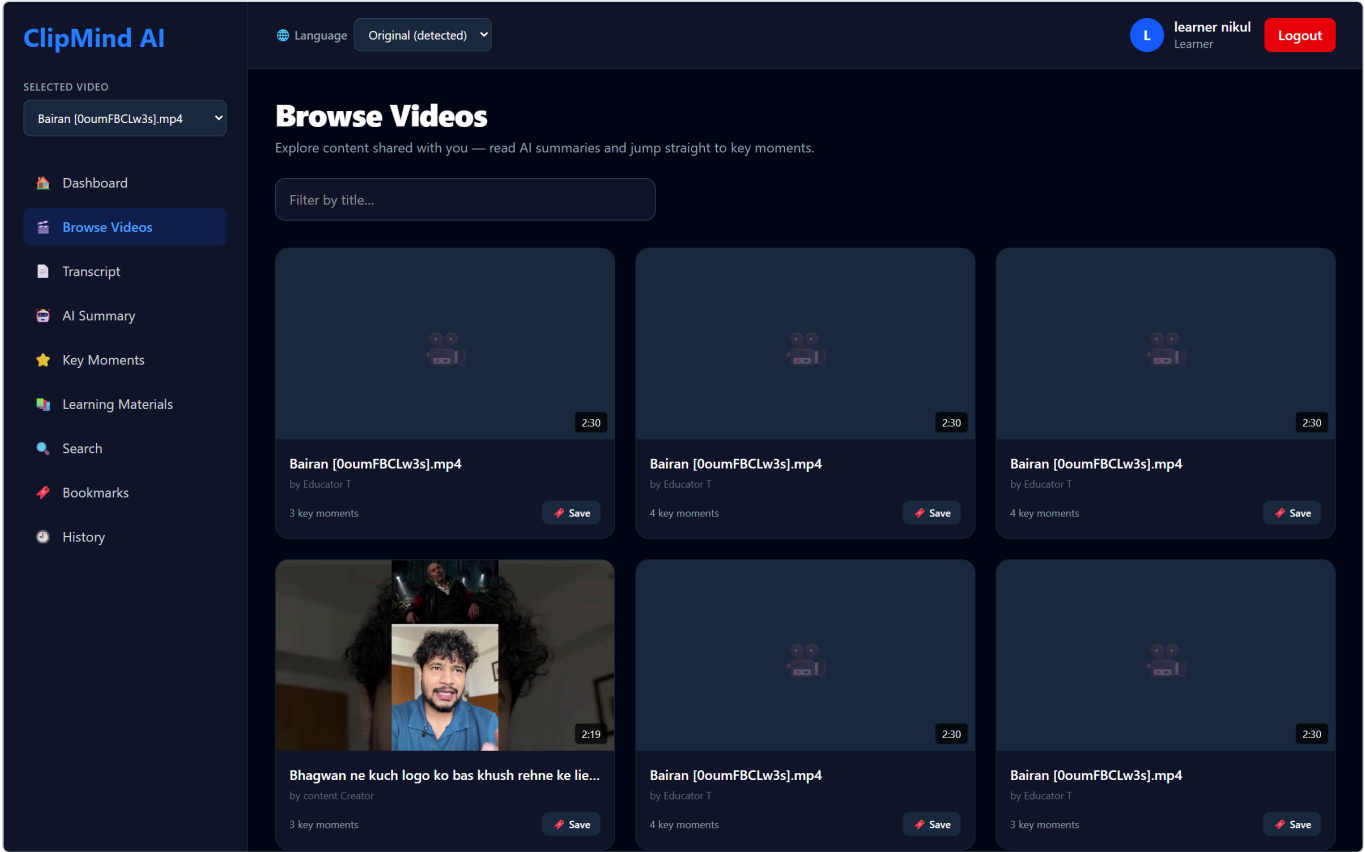


Figure 10.12 — The shared library: completed videos other users have shared, excluding the learner's own uploads.

10.7 Sharing beyond the platform

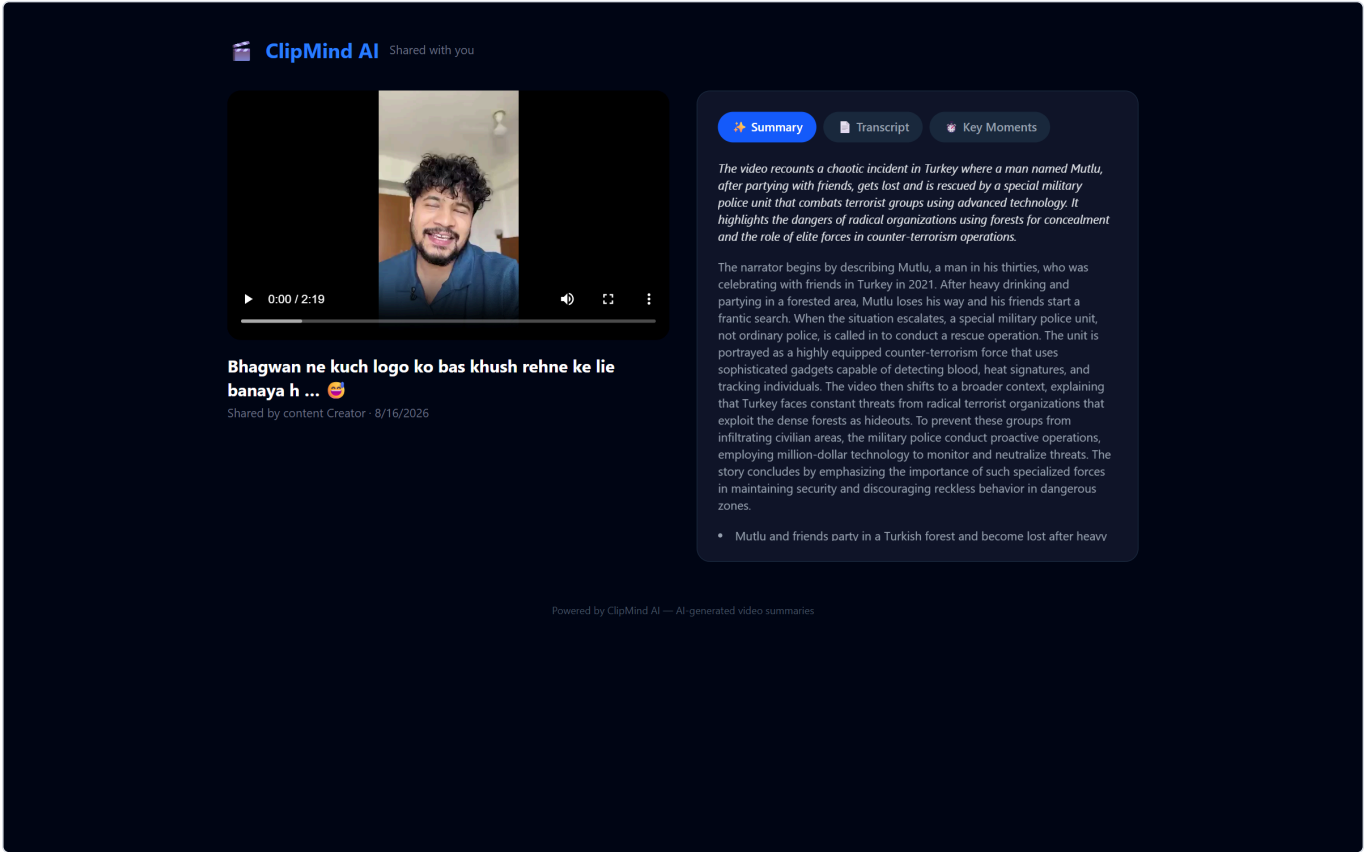


Figure 10.13 — The public share page, captured in a clean browser session with **no authentication**. Only presentation data is exposed — no owner e-mail, no processing internals, no error messages. Revoking the share invalidates this link immediately.

10.8 Analytics for creators

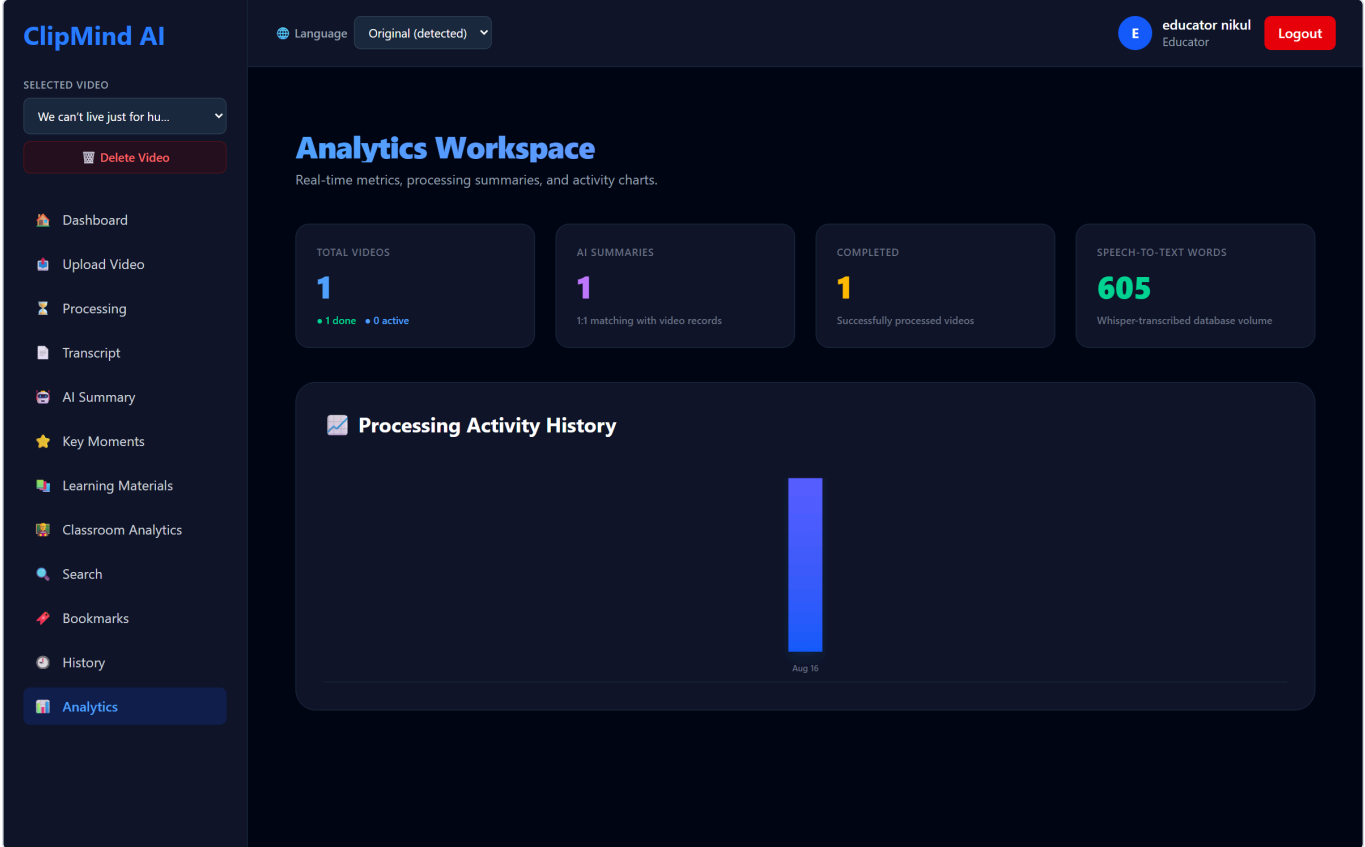


Figure 10.14 — Personal and content analytics, driven by `/analytics/` and `/analytics/content`.

10.9 The administrator console

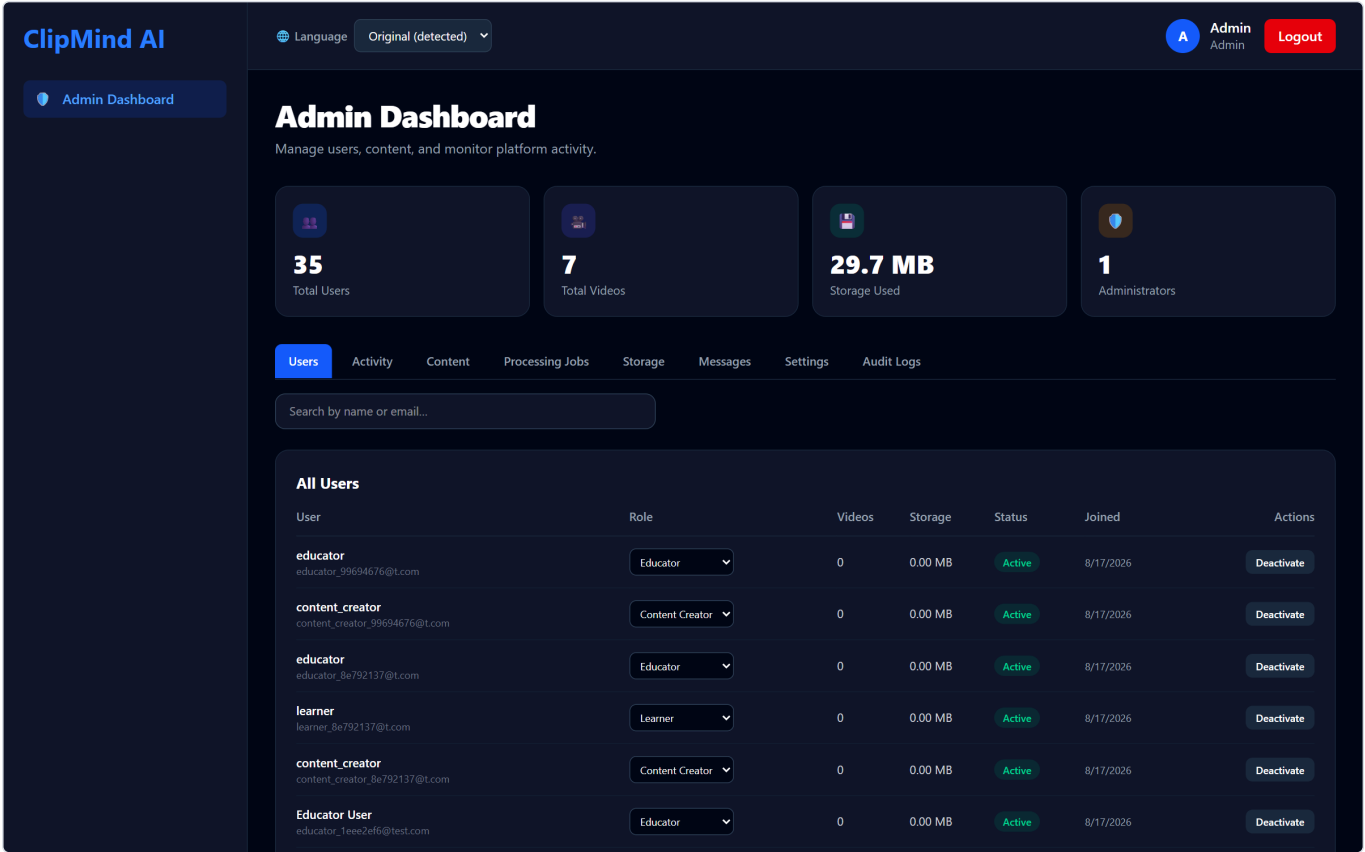


Figure 10.15 — User and role management. Role changes and deactivations write audit entries; an administrator cannot demote or deactivate their own account.

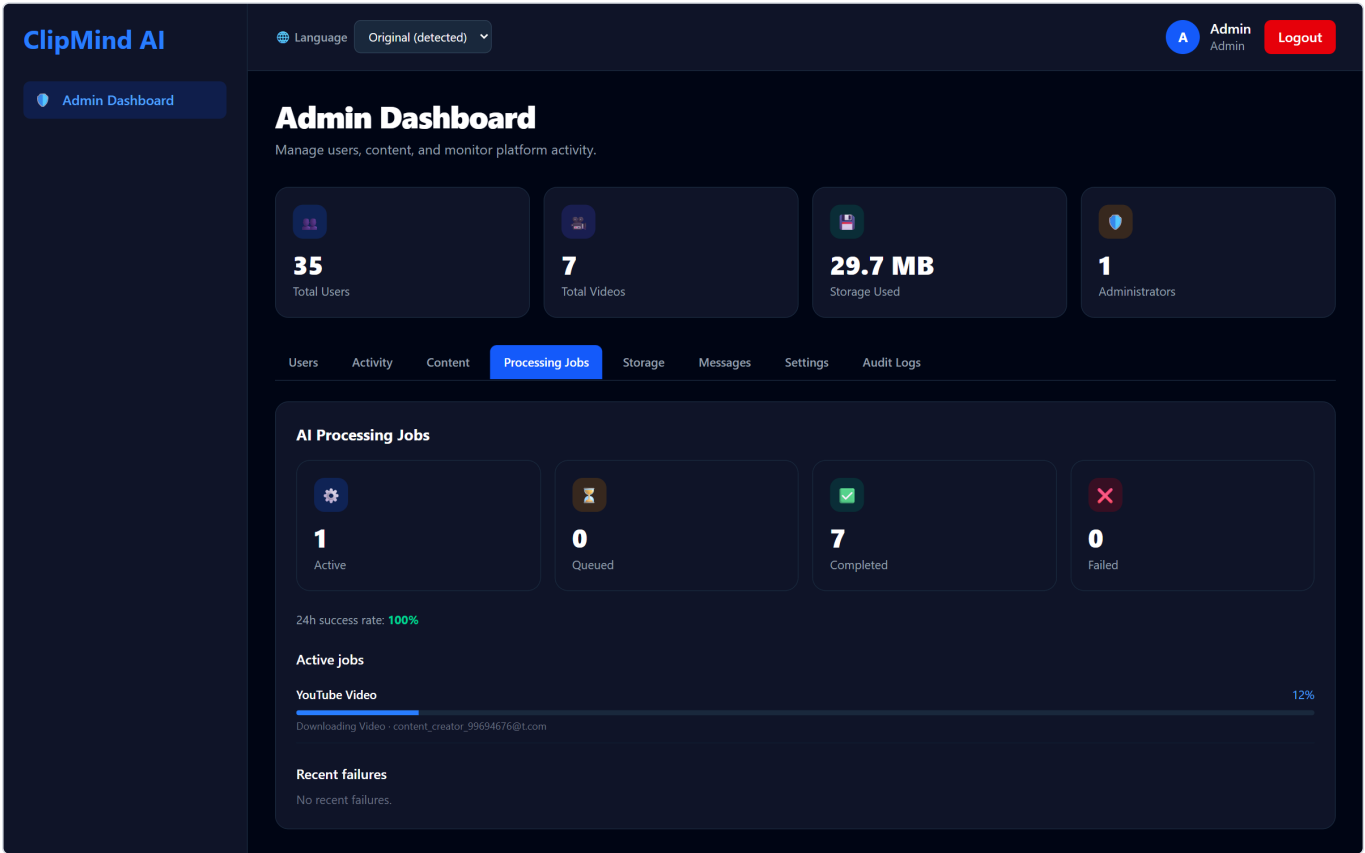


Figure 10.16 — The AI job monitor, captured with a job genuinely in flight ("Downloading Video", 12 %). Because pipeline state lives on the `videos` row, this view needs no separate job table.

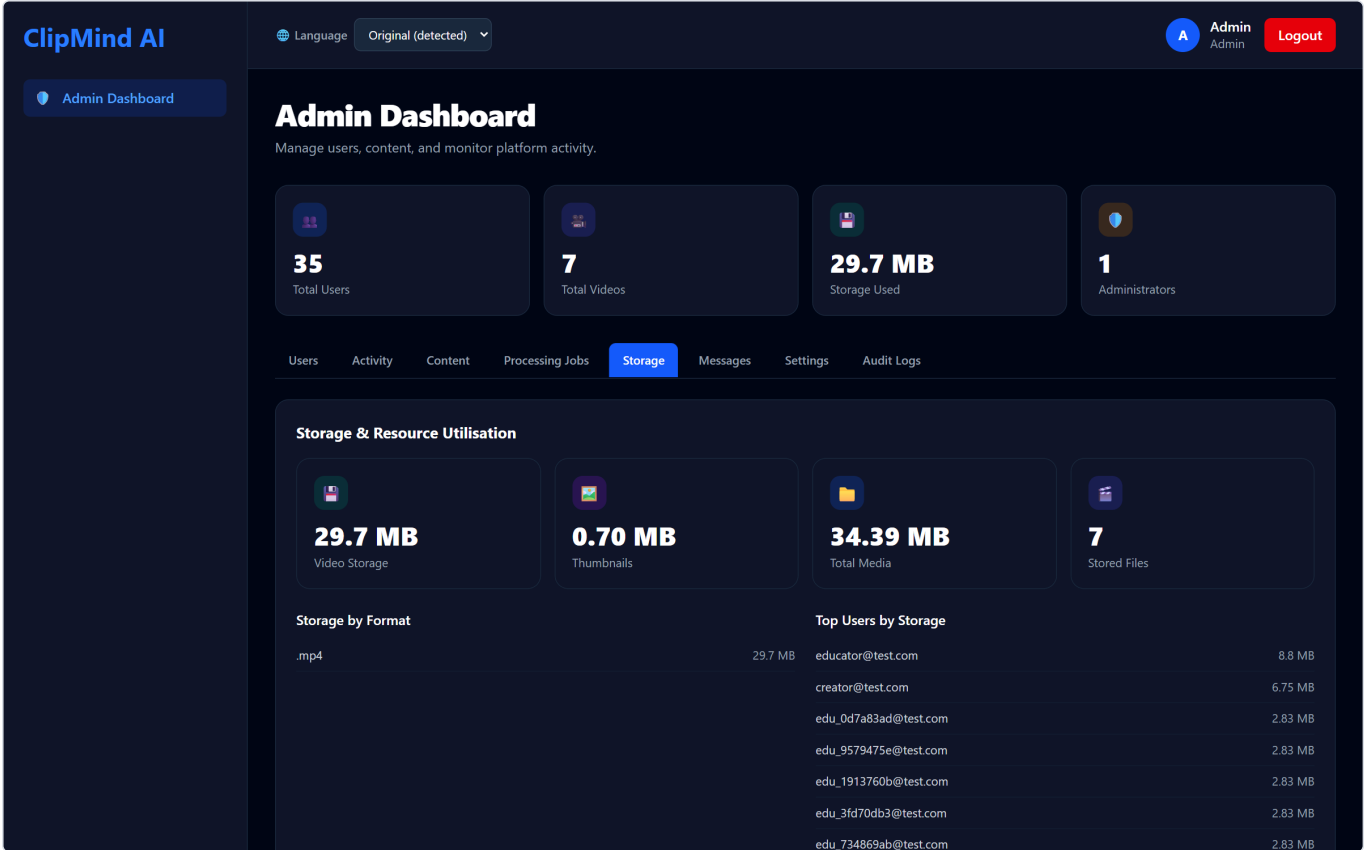


Figure 10.17 — Storage and resource utilisation, computed by walking `MEDIA_ROOT` and summing per-user and per-format footprints.

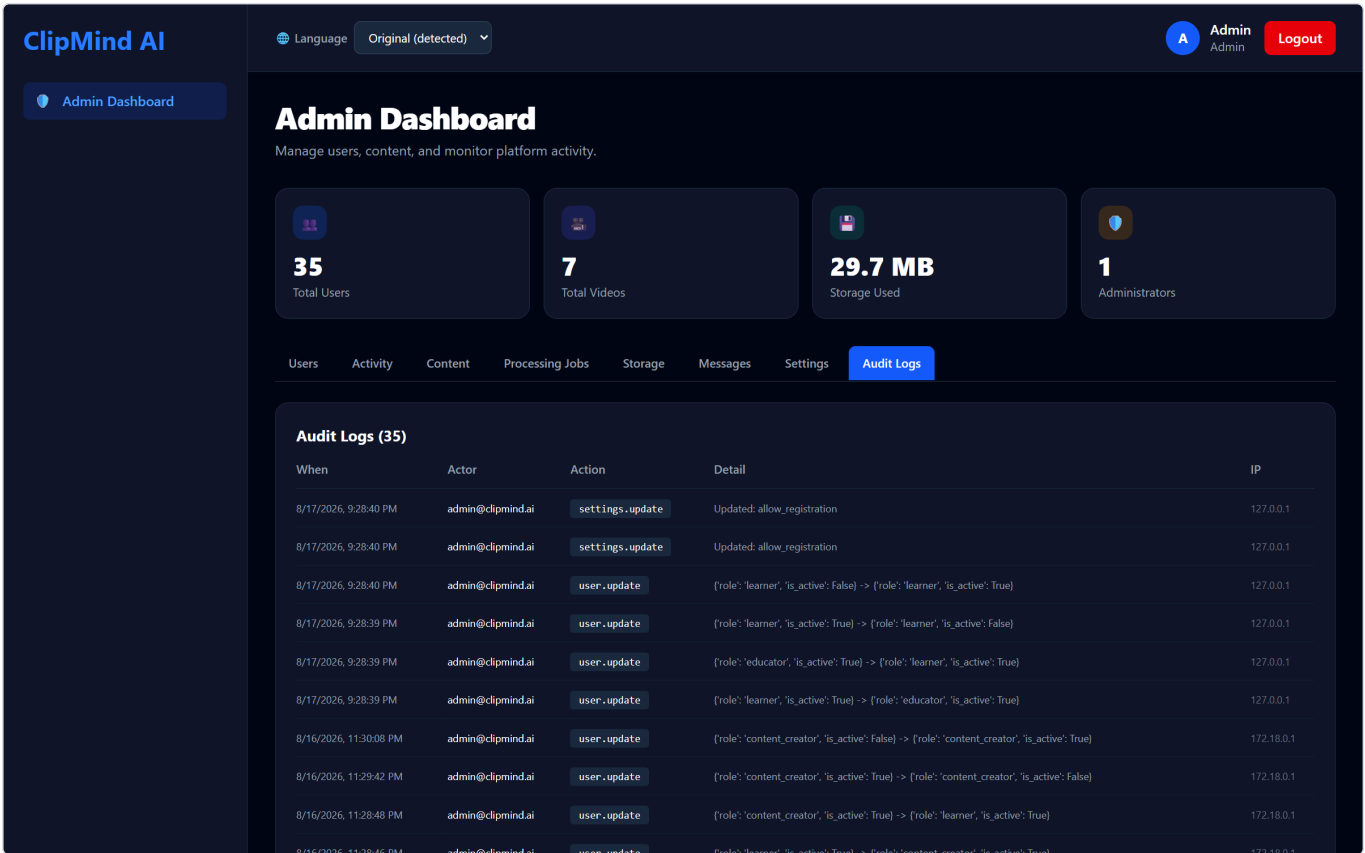


Figure 10.18 — The audit trail. `actor_email` is denormalised onto each row so the record survives deletion of the administrator's account.

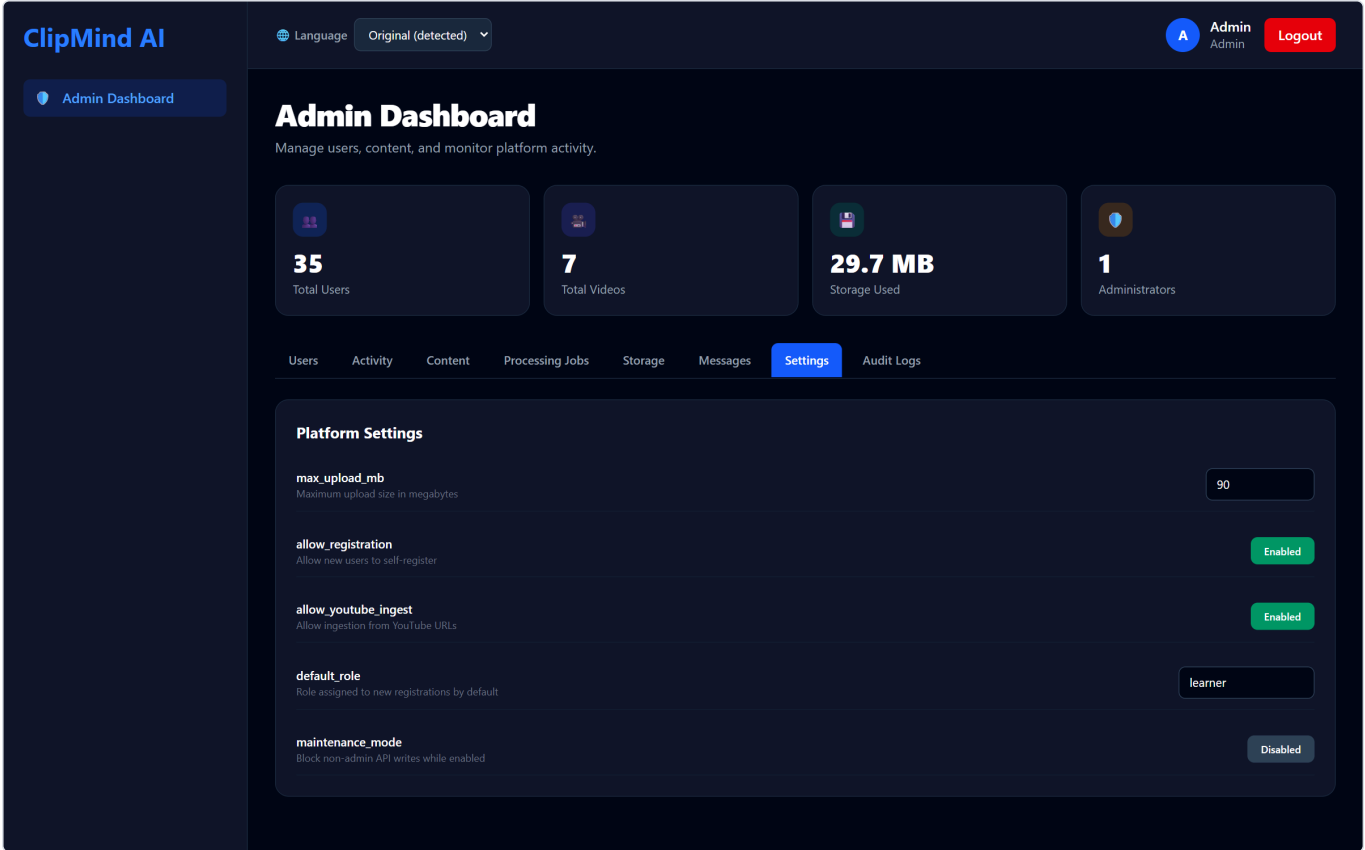


Figure 10.19 — Platform settings. Only the five whitelisted keys are accepted, a type change is rejected, and every successful update is audited.

10.10 Generated API documentation

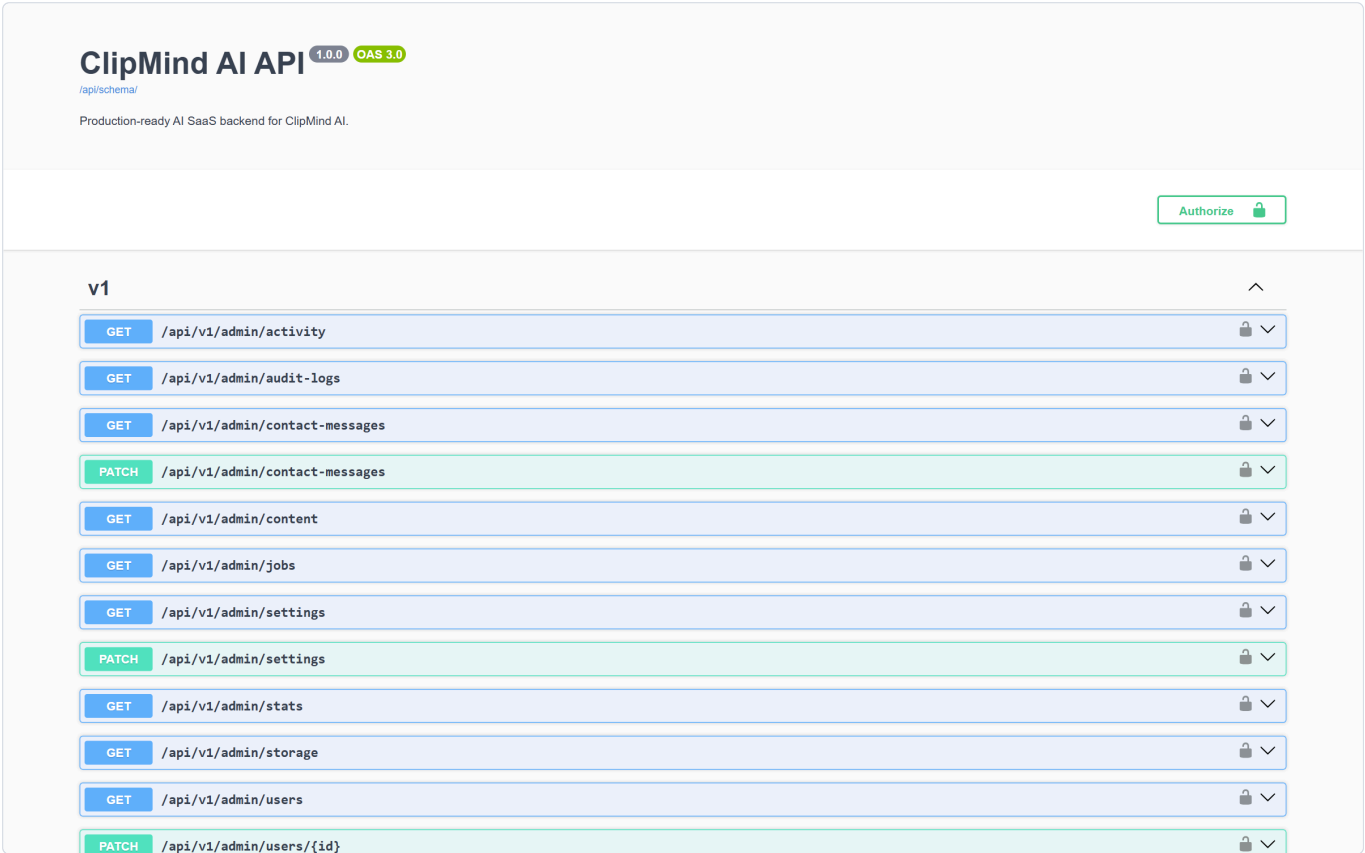


Figure 10.20 — Swagger UI at `/api/docs/`, generated by drf-spectacular from the live URL configuration and serialisers. ReDoc is available at `/api/redoc/` and the raw schema at `/api/schema/`.

11. Security Implementation

Controls are listed by layer, each with the file that implements it.

11.1 Configuration and transport

Control	Implementation
No hardcoded secrets	Every secret is read through <code>python-decouple</code> ; <code>.env</code> is git-ignored and only <code>.env.example</code> is committed.
Environment-aware hardening	<code>SECURE_SSL_REDIRECT</code> , <code>SESSION_COOKIE_SECURE</code> , <code>CSRF_COOKIE_SECURE</code> , HSTS (1 year, subdomains, preload) all key off <code>DEBUG</code> , so they switch on automatically in production without a second settings file.
TLS behind a proxy	<code>SECURE_PROXY_SSL_HEADER</code> + <code>USE_X_FORWARDED_HOST</code> , which prevents the redirect loop that occurs when TLS terminates at a load balancer.
Clickjacking / MIME sniffing	<code>X_FRAME_OPTIONS = "DENY"</code> , <code>SECURE_CONTENT_TYPE_NOSNIFF</code> , <code>SECURE_REFERRER_POLICY = "same-origin"</code> .
CORS & CSRF	Explicit origin allow-lists from environment; <code>CSRF_COOKIE_HTTPONLY=False</code> is a deliberate exception so the SPA can read the token.

11.2 Authentication and authorisation

Control	Implementation
Short-lived access tokens	60-minute access token; 7-day refresh with rotation and blacklisting, so a leaked refresh token dies the moment the real client refreshes.
Password strength	Regex policy plus Django's four validators; PBKDF2-SHA256 hashing (MD5 is used only inside the test database, where it never touches real credentials).
Privilege escalation blocked	<code>admin</code> excluded from <code>SELF_ASSIGNABLE_ROLES</code> and re-validated in <code>validate_role()</code> .
Administrator lockout prevention	Self-deactivation and self-demotion are explicitly refused before the serialiser runs.
Account enumeration	Password reset always returns the same message; unauthorised object reads return 404 rather than 403.
Accountability	Every privileged action writes an <code>AuditLog</code> row with the actor, the target, a before/after detail string and the client IP (X-Forwarded-For aware).

11.3 Upload and media

Control	Implementation
Content-type spoofing	<code>validate_video_file()</code> reads the first 12 bytes and checks for ISO-BMFF <code>ftyp</code> , RIFF/AVI or the EBML magic — a renamed executable fails validation even with a <code>.mp4</code> extension. The file pointer is restored afterwards so the upload still saves intact.
Size limits	2 GB hard cap, empty files rejected, and 10 MB in-memory limits force large uploads to stream to disk.
Private media	Signed, path-bound, 12-hour tokens; three-way authorisation in <code>_authorize_media()</code> ; unauthorised access returns 404, never 403.
Path traversal	<code>serve_media</code> resolves the absolute path and rejects anything not strictly inside <code>MEDIA_ROOT</code> .
Container privilege	The Docker image creates and switches to a non-root <code>appuser</code> — the process that handles untrusted uploaded media does not run as root.

11.4 Abuse and data exposure

- **Throttling** — 100/day anonymous, 1000/day authenticated, and a dedicated 5/hour scope on the contact form, which is both unauthenticated and capable of sending e-mail.
- **Restricted public payload** — `PublicSharedVideoSerializer` exposes only presentation fields; owner e-mail, processing internals and error text are absent by construction rather than by filtering.
- **Unguessable tokens** — share tokens and reset tokens are both `secrets.token_urlsafe(32)`.
- **Log hygiene** — `utils/helpers.mask_email()` exists for safe logging, and the audit trail stores IPs rather than raw request bodies.
- **SQL injection** — all data access goes through the Django ORM; there is no raw SQL anywhere in the application code.

Known gaps, stated honestly. There is no e-mail verification flow despite the `is_verified` field existing; no account lockout after repeated failed logins (only global throttling); tokens live in `localStorage` rather than httpOnly cookies, which is a deliberate trade-off for a cross-origin SPA but does mean an XSS bug would expose them; and media is served by Django rather than by a CDN with signed URLs, which is documented in `core/media.py` as acceptable for this scale.

12. Deployment & Operations

12.1 Compose topology

`docker compose up -d --build` brings up five services. The frontend build context points at the sibling repository, so both projects must sit side by side in the same parent directory.

Service	Image	Port	Notes
db	postgres:16-alpine	5433+5432	<code>pg_isready</code> health check; named volume <code>postgres_data</code> .
redis	redis:7-alpine	6380+6379	AOF persistence enabled; <code>redis-cli ping</code> health check.
backend	built from <code>./Dockerfile</code>	8000	Gunicorn, 3 workers, 120 s timeout. <code>RUN_MIGRATIONS=true</code> ; curl health check against <code>/health/</code> with a 40 s start period.
worker	same image	—	<code>celery -A config worker --concurrency=2</code> . Inherits the backend environment via a YAML anchor but overrides <code>RUN_MIGRATIONS=false</code> so two containers never race to migrate. Shares the media volume.
frontend	built from <code>../clip_mind_AI</code>	3000+80	Multi-stage: <code>node:20-alpine</code> builds, <code>nginx:1.27-alpine</code> serves. <code>VITE_API_URL</code> is a build arg because Vite inlines env vars at build time.

Volumes: `postgres_data` , `redis_data` , `media_data` (shared between backend and worker), `logs_data` . Host ports 5433 and 6380 are used for Postgres and Redis to avoid colliding with locally installed instances.

12.2 Backend image

Based on `python:3.12-slim` . It installs `ffmpeg` (compression, audio extraction, thumbnails), `libpq5` (required by `psycopg2-binary`) and `curl` (health check), copies `requirements.txt` first so the dependency layer caches across code changes, normalises CRLF line endings on the entrypoint (the repository is edited on Windows, and a CRLF shebang makes the kernel fail to find the interpreter), creates the writable runtime directories, and finally switches to a non-root user.

12.3 Entrypoint sequence

- Poll the database host/port with a small Python socket loop until it accepts connections.
- If `RUN_MIGRATIONS=true` : `migrate --noinput` , then `collectstatic --noinput --clear` .
- If `DJANGO_SUPERUSER_EMAIL` and `DJANGO_SUPERUSER_PASSWORD` are both set, create that administrator only if it does not already exist — idempotent, safe on every boot.
- `exec "$@"` so the real process (Gunicorn or Celery) becomes PID 1 and receives signals correctly.

12.4 nginx configuration

Listens on both IPv4 and IPv6 (inside the container `localhost` resolves to `::1` first, so an IPv4-only listener makes health probes fail), gzips text assets, caches hashed `/assets/` for a year as immutable, serves `index.html` with `no-store` so a redeploy is picked up immediately, and falls back to `index.html` for unknown paths so a hard refresh on `/shared/<token>` or `/dashboard` does not 404.

12.5 Operational commands

```

docker compose --env-file .env.docker up -d --build      # start the whole stack
docker compose --env-file .env.docker ps                # service status + health
docker compose --env-file .env.docker logs -f worker    # watch the AI pipeline live
docker compose --env-file .env.docker exec backend python manage.py test
docker compose --env-file .env.docker down              # stop (keeps volumes)
docker compose --env-file .env.docker down -v           # stop and wipe the database

python manage.py cleanup_orphan_media --dry-run          # list unreferenced media files
python manage.py cleanup_orphan_media                   # and delete them

```

12.6 Local development without Docker

Requires PostgreSQL, a Redis-protocol server (Memurai on Windows) and FFmpeg. Three processes are needed: `python manage.py runserver` , `celery -A config worker` `--pool=solo -l info` (the `solo` pool is mandatory on Windows, where the prefork pool is unsupported) and `npm run dev` on port 5173. If the worker is not running, videos remain stuck at `pending` forever — a symptom documented explicitly in the project's own `RUNNING.md` .

12.7 Verified endpoints of a running stack

URL	Serves
<code>http://localhost:3000</code>	React SPA via nginx.
<code>http://localhost:8000</code>	Django REST API.
<code>http://localhost:8000/api/docs/</code>	Swagger UI.
<code>http://localhost:8000/api/redoc/</code>	ReDoc.
<code>http://localhost:8000/admin/</code>	Django admin (User and ContactMessage registered).
<code>http://localhost:8000/health/</code>	Health probe — verified returning <code>{"status": "healthy", "checks": {"database": "ok", "cache": "ok"}}</code> .

13. Configuration Reference

Configuration is entirely environment-driven. Every variable below has a safe default in `settings.py`, so the project starts without a fully populated `.env` — but the AI pipeline requires at least one provider key.

Variable	Default	Effect
SECRET_KEY	change-me-in-production	Django cryptographic signing, including media tokens.
DEBUG	False	Master switch for the entire production-hardening block.
ALLOWED_HOSTS	127.0.0.1,localhost	Host header allow-list (CSV).
DB_NAME / DB_USER / DB_PASSWORD / DB_HOST / DB_PORT	clipmind_ai / postgres / - / localhost / 5432	PostgreSQL connection.
REDIS_URL	redis://localhost:6379/0	Celery broker and result backend.
REDIS_CACHE_URL	redis://localhost:6379/1	Django cache — translations and reset tokens.
JWT_SECRET	falls back to SECRET_KEY	Token signing key, separable from the Django secret.
GROQ_API_KEY	""	Primary AI provider. Without it (and without a Gemini key) provider initialisation raises <code>AIProviderError</code> and every pipeline run fails.
GEMINI_API_KEY	""	Fallback provider; also usable as the sole provider.
CORS_ALLOWED_ORIGINS	localhost:3000, 127.0.0.1:3000	Browser origins allowed to call the API with credentials.
CSRF_TRUSTED_ORIGINS	localhost:3000, :5173, 127.0.0.1:3000	Origins trusted for CSRF.
FRONTEND_URL	http://localhost:5173	Base URL used to build password-reset links and <code>share_url</code> . A mismatch here is why a share link can point at the wrong port.
EMAIL_HOST / EMAIL_PORT / EMAIL_HOST_USER / EMAIL_HOST_PASSWORD	smtp.gmail.com / 587 / - / -	SMTP with TLS for reset and contact mail.
DEFAULT_FROM_EMAIL / CONTACT_EMAIL	sender / falls back to sender	Outbound sender and contact-form destination.
SECURE_SSL_REDIRECT	not DEBUG	Overridable so a container behind a TLS-terminating proxy can disable it.
RUN_MIGRATIONS	false	Entrypoint-only: which container owns schema migration.
DJANGO_SUPERUSER_EMAIL / _PASSWORD / _FIRSTNAME	- / - / Admin	Optional first-administrator seeding on boot.
VITE_API_URL	http://localhost:8000	Frontend, build-time. Vite inlines it into the bundle, so changing it requires a rebuild.
VITE_CONTACT_EMAIL	placeholder shown verbatim	Contact address displayed on the public site; left as an obvious placeholder until configured.

13.1 Runtime constants worth knowing

Constant	Value	Defined in
Maximum upload size	2 GB	<code>utils/validators.py</code>
Accepted containers	mp4, mov, avi, mkv, webm	<code>utils/validators.py</code>
Compression target	CRF 28, max width 1280	<code>apps/processing/tasks.py</code>
Transcript sent to the LLM	14,000 characters	<code>services/ai/groq_provider.py</code>
Translation batch size	40 segments	<code>apps/videos/services.py</code>
Translation cache TTL	7 days	<code>apps/videos/services.py</code>
Media token TTL	12 hours	<code>core/media.py</code>
Password reset TTL	30 minutes	<code>apps/accounts/services.py</code>
Celery task time limit	1 hour	<code>config/settings.py</code>
Search match cap	20 per video	<code>apps/videos/views.py</code>
Completion threshold	90 % watched	<code>apps/videos/views.py</code>

14. Testing & Validation

142

TESTS

142

PASSING

0

FAILURES

8.7 s

RUNTIME

22

TEST CLASSES

14.1 Execution evidence

Run inside the live backend container against a throwaway test database:

```
$ docker compose exec backend python manage.py test

Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 142 tests in 8.720s

OK
Destroying test database for alias 'default'...
```

14.2 Suite composition

Module	Tests	What it establishes
apps/videos/tests.py	68	Nine classes covering upload permissions, the administrator-is-not-an-author boundary, per-user video isolation, the full sharing model (public, restricted, revocation, re-share token stability), bookmarks and history, transcript editing, cross-video search, learning materials and signed-media security.
apps/accounts/tests.py	39	Registration (including that <code>admin</code> cannot be self-assigned and weak passwords are refused), login and deactivated accounts, role capability flags, private activity history, the throttled contact form, and every administrator endpoint including self-lockout prevention and settings type validation.
apps/processing/tests.py	19	YouTube-ID extraction across every URL shape, analysis normalisation, the end-to-end pipeline with mocked providers, timestamp clamping, failure recording, and an AI output quality class.
apps/analytics/tests.py	9	Personal metric accuracy and per-user isolation, topic aggregation from AI summaries, view/bookmark counting, educator-only classroom access, and the exclusion of the educator's own views.

14.3 AI output quality validation

Most projects stop at "the endpoint returned 200". `AIQualityValidationTests` goes further and asserts properties of the generated content itself:

- the summary is shorter than the transcript, and the short summary shorter than the detailed one;
- every key-moment timestamp lies within the video's real duration;
- key moments are ordered and their timestamps unique;
- all required analysis sections are populated;
- glossary entries and question/answer pairs are well-formed;
- transcript segments are monotonic and correctly timed;
- the plain-text transcript matches the concatenation of its segments.

14.4 Test-run configuration

`settings.py` detects `manage.py test` and applies four overrides that make the suite fast and hermetic: MD5 password hashing (PBKDF2 costs ~0.5 s per hash and every `setUp` creates users — and this applies only to the throwaway test database), eager Celery execution so no worker or broker is needed, a local-memory cache instead of Redis, media redirected to `test_media/`, throttling disabled (otherwise repeated API calls would 429) and `SECURE_SSL_REDIRECT` forced off.

14.5 Manual and integration validation performed for this document

Check	Result
Full stack starts via Docker Compose	verified — all five containers reported healthy.
<code>GET /health/</code>	200 <code>{"status":"healthy","checks":{"database":"ok","cache":"ok"}}</code>
OpenAPI schema generation	verified — 40 paths produced, Swagger UI renders.
Login through the real API	verified for administrator, educator, creator and learner accounts.
End-to-end AI artefacts present in the database	verified — 7 videos, transcripts, 8-section summaries, key moments with rendered thumbnails, 25 learning materials, 6 shares, 12 history rows.
Public share link without authentication	verified — rendered in a clean browser session (Figure 10.13).
Live job monitoring	verified — an in-flight pipeline job displayed at 12 % (Figure 10.16).
Cross-video search returning timestamped matches	verified — Devanagari query matched two segments (Figure 10.8).

15. Limitations, Trade-offs & Future Work

15.1 Deliberate trade-offs

Decision	Cost	Why it was made
Polling instead of WebSockets	Up to 3 s of latency on a status change; one extra request per active video per 3 s.	Keeps the deployment to a single WSGI process type. The endpoint is tiny and polling stops the moment the status is terminal.
Summary stored as a JSON blob	Cannot be queried at the SQL level; topic aggregation happens in Python.	The LLM output schema can evolve without a migration, and the whole analysis is written and read in one operation.
Transcript truncated at 14,000 characters	Very long videos are summarised from their first portion only.	The free Groq tier caps requests at roughly 8,000 tokens per minute; prompt plus a 4,096-token completion must fit inside it.
Local filesystem media	Does not scale horizontally without shared storage; Django serves the bytes.	Explicitly documented in <code>core/media.py</code> as appropriate for this scale, with nginx/S3/CloudFront named as the production path.
Full re-encode of every video	Significant CPU time per upload.	Guarantees a seekable, uniformly sized, browser-playable H.264 file regardless of what was uploaded, and reclaims storage by deleting the original.
Tokens in <code>localStorage</code>	Readable by any script running on the page.	Required for a cross-origin SPA that attaches a bearer header; the alternative (httpOnly cookies) would demand same-site hosting and CSRF handling on every request.

15.2 Known limitations

- No e-mail verification workflow, despite `is_verified` existing on the user model.
- No Google OAuth, despite `login_provider` existing as a field.
- `deleted_at` exists but soft deletion is not implemented — deletions are hard.
- Failed pipeline runs cannot be retried from the UI; the video must be re-submitted.
- The `maintenance_mode` platform setting is stored and editable but not yet enforced by any middleware.
- No frontend unit or end-to-end test suite; the client is validated by ESLint and manual testing only.
- Analytics aggregate in Python for file sizes and topic clouds, which will not scale to very large libraries without materialised counters.
- YouTube ingestion depends on `yt-dlp` and a specific player-client workaround, which is inherently fragile against upstream changes.

15.3 Natural next steps

1. **Chunked summarisation** — map/reduce over transcript windows to remove the 14,000-character ceiling.
2. **Object storage** — move `MEDIA_ROOT` to S3 with CloudFront signed URLs, reusing the existing signed-URL abstraction almost unchanged.
3. **Retry controls** — expose Celery retry from the admin job monitor for transient provider failures.
4. **Semantic search** — embed transcript segments and add vector similarity alongside the existing literal match.
5. **Frontend test suite** — Vitest plus React Testing Library for the contexts and guards, which carry the most logic.
6. **Enforce `maintenance_mode`** — a small middleware closing the loop on a setting that is already exposed in the UI.

15.4 Assessment

Measured against what it sets out to do, the implementation is complete and internally consistent. The role model is not decorative: it is enforced twice, tested exhaustively, and the awkward cases (an administrator who may moderate but not author; a learner who may read a shared transcript but never edit it; an educator whose own views must not inflate their classroom statistics) are all handled deliberately. The AI layer is abstracted behind an interface with genuine runtime failover rather than a single hardcoded SDK call. The security posture goes well beyond a student project's usual baseline — magic-byte upload validation, path-bound signed media tokens, refresh-token rotation with blacklisting, 404-instead-of-403 on unauthorised reads, and an audit trail that survives deletion of its actor. Most of the code comments explain *why* a decision was taken rather than restating what the line does, which is the clearest signal that the system was reasoned about rather than assembled.

16. Appendix — Repository Inventory

A. Backend modules

Path	Contents
config/settings.py	391 lines — apps, middleware, database, DRF, SimpleJWT, CORS, cache, Celery, e-mail, security, logging, test overrides.
config/urls.py	Root URLconf: admin, four API includes, public contact, health, schema/docs/redoc, range-capable media.
config/celery.py	Celery app with <code>CELERY_</code> namespace and task autodiscovery.
apps/accounts/	models (5), serialisers (13), <code>AuthService</code> , 9 auth views, 10 admin views, contact view, 5 migrations, 39 tests.
apps/videos/	models (6), serialisers (9), services, 17 views, management command, 6 migrations, 68 tests.
apps/processing/tasks.py	351 lines — the whole pipeline plus its FFmpeg/yt-dlp helpers.
apps/transcripts/ · apps/summaries/	<code>Transcript</code> , <code>Summary</code> , <code>KeyMoment</code> , <code>LearningMaterial</code> .
apps/analytics/views.py	Three aggregate endpoints and the dense daily-series helper.
core/ (8 modules)	responses, permissions, exceptions, <code>exception_handler</code> , media, pagination, health, <code>logging_service</code> .
services/ai/ (4 modules)	base ABC, Groq provider, Gemini provider, fallback provider manager.
utils/	validators (upload magic bytes, YouTube URL, language hints, password policy) and helpers.

B. Frontend modules

Path	Contents
src/pages/ (23)	Home, FeaturesPage, About, Contact, Login, Register, ForgotPassword, ResetPassword, SharedVideo, Dashboard, UploadVideo, Processing, Transcript, AISummary, KeyMoments, Library, Search, Bookmarks, History, LearningMaterials, Analytics, ClassroomAnalytics, AdminDashboard.
src/components/auth/ (6)	AuthLayout, LoginForm, RegisterForm, ForgotPasswordForm, ResetPasswordForm, PasswordInput.
src/components/home/ (5)	Hero, Features, HowItWorks, RoleShowcase, FAQ.
src/components/dashboard/ (2)	Sidebar (role-filtered nav, video picker, delete confirmation) and TopNavbar (language selector, identity, logout).
src/context/ (2)	AuthContext (147 lines), VideoContext (301 lines).
src/lib/ (5)	api, roles, player, motion, site.

C. Project documentation already in the repositories

File	Purpose
README.md	Frontend overview and setup.
RUNNING.md	Docker and local run instructions, everyday commands, troubleshooting table.
DEPLOYMENT.md	Deployment guidance.
TESTING.md	Test documentation.
.env.example · .env.docker.example	Documented configuration templates with no real secrets.

D. Quick-start summary

```
# Both repositories must sit side by side in the same parent directory.
cd clip_mind_AI_Backend
cp .env.docker.example .env.docker      # then set GROQ_API_KEY (and/or GEMINI_API_KEY)
docker compose --env-file .env.docker up -d --build

# Frontend → http://localhost:3000
# API      → http://localhost:8000
# Swagger  → http://localhost:8000/api/docs/
# Health   → http://localhost:8000/health/
```

End of document. Every figure, table, endpoint, constant and test count in this report was taken from the repository source or from a live, running instance of the application.